

CT60A2600-TIIMI-A

Generated by Doxygen 1.9.1

1 Class Index	1
1.1 Class List	1
2 File Index	3
2.1 File List	3
3 Class Documentation	5
3.1 jono Struct Reference	5
3.1.1 Detailed Description	5
3.1.2 Member Data Documentation	5
3.1.2.1 pSeuraava	5
3.1.2.2 pSolmu	5
3.2 puu Struct Reference	6
3.2.1 Detailed Description	6
3.2.2 Member Data Documentation	6
3.2.2.1 aNimi	6
3.2.2.2 iArvo	6
3.2.2.3 iPituus	6
3.2.2.4 pOikea	7
3.2.2.5 pVasen	7
3.3 RBSolmu Struct Reference	7
3.3.1 Detailed Description	7
3.3.2 Member Data Documentation	7
3.3.2.1 aNimi	7
3.3.2.2 iArvo	8
3.3.2.3 iVariBitti	8
3.3.2.4 pOikea	8
3.3.2.5 pVanhempi	8
3.3.2.6 pVasen	8
3.4 tiedot Struct Reference	8
3.4.1 Detailed Description	9
3.4.2 Member Data Documentation	9
3.4.2.1 aSukunimi	9
3.4.2.2 iYhteensa	9
3.4.2.3 pEdellinen	9
3.4.2.4 pSeuraava	9
4 File Documentation	11
4.1 binaaripuu.c File Reference	11
4.1.1 Function Documentation	12
4.1.1.1 etsiPienin()	12
4.1.1.2 kirjoitaBinaaripuu()	12
4.1.1.3 kirjoitaTiedostoon()	13

4.1.1.4 lisaaSolmu()	13
4.1.1.5 luoPuu()	14
4.1.1.6 nimenArvo()	15
4.1.1.7 oikeaPuoli()	16
4.1.1.8 onkoLuku()	16
4.1.1.9 paivitaPuu()	17
4.1.1.10 poistaSolmu()	18
4.1.1.11 puunPituus()	19
4.1.1.12 suurempiLukuVertailu()	19
4.1.1.13 tasapainoitaPuu()	20
4.1.1.14 vapautaMuistiPuu()	20
4.1.1.15 varaaMuistiaPuulle()	21
4.1.1.16 vasenPuoli()	21
4.2 binaaripuu.h File Reference	22
4.2.1 Typedef Documentation	23
4.2.1.1 PUU	23
4.2.1.2 RBSOLMU	23
4.2.2 Function Documentation	24
4.2.2.1 etsiPienin()	24
4.2.2.2 kierraOikealle()	24
4.2.2.3 kierraVasemmalle()	25
4.2.2.4 kirjoitaBinaaripuu()	25
4.2.2.5 kirjoitaRB()	25
4.2.2.6 kirjoitaRBTiedostoon()	26
4.2.2.7 kirjoitaTiedostoon()	26
4.2.2.8 korjaaLisays()	27
4.2.2.9 lisaaRBSolmu()	28
4.2.2.10 lisaaSolmu()	28
4.2.2.11 luoPuu()	29
4.2.2.12 luoRBPuu()	30
4.2.2.13 nimenArvo()	31
4.2.2.14 oikeaPuoli()	31
4.2.2.15 onkoLuku()	32
4.2.2.16 paivitaPuu()	33
4.2.2.17 poistaSolmu()	33
4.2.2.18 puunPituus()	34
4.2.2.19 suurempiLukuVertailu()	35
4.2.2.20 tasapainoitaPuu()	35
4.2.2.21 vapautaMuistiPuu()	36
4.2.2.22 vapautaMuistiRB()	36
4.2.2.23 varaaMuistiaPuulle()	37
4.2.2.24 varaaMuistiaRB()	38

4.2.2.25 vasenPuoli()	38
4.3 haut.c File Reference	39
4.3.1 Function Documentation	39
4.3.1.1 binaarihaku()	40
4.3.1.2 leveyshaku()	40
4.3.1.3 syvyyshaku()	41
4.3.1.4 tarkistaLoytukoSyvyysaulla()	42
4.3.1.5 vapautaMuistiJono()	43
4.3.1.6 varaaMuistiaJonolle()	43
4.4 haut.h File Reference	44
4.4.1 Typedef Documentation	44
4.4.1.1 JONO	44
4.4.2 Function Documentation	44
4.4.2.1 binaarihaku()	44
4.4.2.2 leveyshaku()	45
4.4.2.3 syvyyshaku()	46
4.4.2.4 tarkistaLoytukoSyvyysaulla()	47
4.4.2.5 vapautaMuistiJono()	47
4.4.2.6 varaaMuistiaJonolle()	48
4.5 linkitettylista.c File Reference	48
4.5.1 Function Documentation	49
4.5.1.1 halkaise()	49
4.5.1.2 kirjoitaTiedostoAlustaLoppuun()	50
4.5.1.3 kirjoitaTiedostoLopustaAlkuun()	51
4.5.1.4 laskeListanPituus()	51
4.5.1.5 lisaaListaan()	52
4.5.1.6 lomitua()	53
4.5.1.7 lomituaLajittelu()	54
4.5.1.8 lue()	55
4.5.1.9 poistaListastaLkmPerusteella()	56
4.5.1.10 poistaListastaNimenPerusteella()	57
4.5.1.11 useamallaAlkiollaSamaLKM()	57
4.5.1.12 vapautaMuisti()	58
4.5.1.13 varaaMuistia()	58
4.6 linkitettylista.h File Reference	59
4.6.1 Typedef Documentation	60
4.6.1.1 TIEDOT	60
4.6.2 Function Documentation	60
4.6.2.1 halkaise()	60
4.6.2.2 kirjoitaTiedostoAlustaLoppuun()	61
4.6.2.3 kirjoitaTiedostoLopustaAlkuun()	62
4.6.2.4 laskeListanPituus()	62

4.6.2.5 lisaaListaan()	63
4.6.2.6 lomitusta()	64
4.6.2.7 lomitustaLajittelu()	65
4.6.2.8 lue()	66
4.6.2.9 poistaListastaLkmPerusteella()	67
4.6.2.10 poistaListastaNimenPerusteella()	68
4.6.2.11 useammallaAlkiollaSamaLKM()	68
4.6.2.12 vapautaMuisti()	69
4.6.2.13 varaaMuistia()	69
4.7 paaohjelma.c File Reference	70
4.7.1 Function Documentation	70
4.7.1.1 main()	71
4.8 punamustapuu.c File Reference	71
4.8.1 Function Documentation	71
4.8.1.1 kierraOikealle()	72
4.8.1.2 kierraVasemmalle()	72
4.8.1.3 kirjoitaRB()	72
4.8.1.4 kirjoitaRBTiedostoon()	73
4.8.1.5 korjaaLisays()	73
4.8.1.6 lisaaRBSolmu()	74
4.8.1.7 luoRBPuu()	75
4.8.1.8 vapautaMuistiRB()	75
4.8.1.9 varaaMuistiaRB()	76
4.9 yleiset.c File Reference	76
4.9.1 Function Documentation	76
4.9.1.1 binaaripuuValikko()	77
4.9.1.2 kysyArvo()	77
4.9.1.3 kysyNimi()	78
4.9.1.4 listaValikko()	78
4.9.1.5 mainBinaaripuu()	79
4.9.1.6 mainLista()	80
4.9.1.7 valikko()	81
4.10 yleiset.h File Reference	81
4.10.1 Macro Definition Documentation	82
4.10.1.1 LEN	82
4.10.2 Function Documentation	82
4.10.2.1 binaaripuuValikko()	82
4.10.2.2 kysyArvo()	83
4.10.2.3 kysyNimi()	83
4.10.2.4 listaValikko()	84
4.10.2.5 mainBinaaripuu()	84
4.10.2.6 mainLista()	85

4.10.2.7 valikko()	86
--------------------	----

Chapter 1

Class Index

1.1 Class List

Here are the classes, structs, unions and interfaces with brief descriptions:

jono		5
puu		6
RBSolmu		7
tiedot		8

Chapter 2

File Index

2.1 File List

Here is a list of all files with brief descriptions:

binaaripuu.c	11
binaaripuu.h	22
haut.c	39
haut.h	44
linkitettylista.c	48
linkitettylista.h	59
paaohjelma.c	70
punamustapuu.c	71
yleiset.c	76
yleiset.h	81

Chapter 3

Class Documentation

3.1 jono Struct Reference

```
#include <haut.h>
```

Public Attributes

- [PUU](#) * [pSolmu](#)
- struct [jono](#) * [pSeuraava](#)

3.1.1 Detailed Description

Definition at line 5 of file haut.h.

3.1.2 Member Data Documentation

3.1.2.1 pSeuraava

```
struct jono* jono::pSeuraava
```

Definition at line 7 of file haut.h.

3.1.2.2 pSolmu

```
PUU* jono::pSolmu
```

Definition at line 6 of file haut.h.

The documentation for this struct was generated from the following file:

- [haut.h](#)

3.2 puu Struct Reference

```
#include <binaaripuu.h>
```

Public Attributes

- char [aNimi](#) [[LEN](#)]
- int [iArvo](#)
- int [iPituus](#)
- struct [puu](#) * [pOikea](#)
- struct [puu](#) * [pVasen](#)

3.2.1 Detailed Description

Definition at line 5 of file binaaripuu.h.

3.2.2 Member Data Documentation

3.2.2.1 aNimi

```
char puu::aNimi [LEN]
```

Definition at line 6 of file binaaripuu.h.

3.2.2.2 iArvo

```
int puu::iArvo
```

Definition at line 7 of file binaaripuu.h.

3.2.2.3 iPituus

```
int puu::iPituus
```

Definition at line 8 of file binaaripuu.h.

3.2.2.4 pOikea

```
struct puu* puu::pOikea
```

Definition at line 9 of file binaaripuu.h.

3.2.2.5 pVasen

```
struct puu* puu::pVasen
```

Definition at line 10 of file binaaripuu.h.

The documentation for this struct was generated from the following file:

- [binaaripuu.h](#)

3.3 RBSolmu Struct Reference

```
#include <binaaripuu.h>
```

Public Attributes

- char [aNimi](#) [[LEN](#)]
- int [iArvo](#)
- int [iVariBitti](#)
- struct [RBSolmu](#) * [pOikea](#)
- struct [RBSolmu](#) * [pVasen](#)
- struct [RBSolmu](#) * [pVanhempi](#)

3.3.1 Detailed Description

Definition at line 13 of file binaaripuu.h.

3.3.2 Member Data Documentation

3.3.2.1 aNimi

```
char RBSolmu::aNimi [LEN]
```

Definition at line 14 of file binaaripuu.h.

3.3.2.2 iArvo

```
int RBSolmu::iArvo
```

Definition at line 15 of file binaaripuu.h.

3.3.2.3 iVariBitti

```
int RBSolmu::iVariBitti
```

Definition at line 16 of file binaaripuu.h.

3.3.2.4 pOikea

```
struct RBSolmu* RBSolmu::pOikea
```

Definition at line 17 of file binaaripuu.h.

3.3.2.5 pVanhempi

```
struct RBSolmu* RBSolmu::pVanhempi
```

Definition at line 19 of file binaaripuu.h.

3.3.2.6 pVasen

```
struct RBSolmu* RBSolmu::pVasen
```

Definition at line 18 of file binaaripuu.h.

The documentation for this struct was generated from the following file:

- [binaaripuu.h](#)

3.4 tiedot Struct Reference

```
#include <linkitettylista.h>
```


Public Attributes

- char [aSukunimi](#) [[LEN](#)]
- int [iYhteensa](#)
- struct [tiedot](#) * [pSeuraava](#)
- struct [tiedot](#) * [pEdellinen](#)

3.4.1 Detailed Description

Definition at line 5 of file linkitettylista.h.

3.4.2 Member Data Documentation

3.4.2.1 aSukunimi

```
char tiedot::aSukunimi[LEN]
```

Definition at line 6 of file linkitettylista.h.

3.4.2.2 iYhteensa

```
int tiedot::iYhteensa
```

Definition at line 7 of file linkitettylista.h.

3.4.2.3 pEdellinen

```
struct tiedot* tiedot::pEdellinen
```

Definition at line 9 of file linkitettylista.h.

3.4.2.4 pSeuraava

```
struct tiedot* tiedot::pSeuraava
```

Definition at line 8 of file linkitettylista.h.

The documentation for this struct was generated from the following file:

- [linkitettylista.h](#)

Chapter 4

File Documentation

4.1 binaaripuu.c File Reference

```
#include "binaaripuu.h"
#include <ctype.h>
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- [PUU * varaaMuistiaPuulle](#) (char *pSolmu, int iArvo)
Muistin varaaminen puu structin alkioille.
- [PUU * vapautaMuistiPuu](#) (PUU *pJuuriSolmu)
Muistin vapauttaminen puu structin alkioista Juurisolmun ja kaikkien sen lapsie solmujen muisti vapautetaan.
- [PUU * lisaaSolmu](#) (PUU *pAlku, char *pSolmu, int iArvo)
Lisaa solmun puuhun.
- [PUU * luoPuu](#) (char *pNimi, [PUU](#) *pJuuriSolmu)
Luetaan tiedosto ja luodaan puu rakenne.
- void [kirjoitaBinaaripuu](#) (char *pNimi, [PUU](#) *pAlku)
Tasapainoitettun binaaripuun kirjoittaminen tiedostoon.
- void [kirjoitaTiedostoon](#) (char *pNimi, [PUU](#) *pAlku)
Kirjoittaa binaaripuun tiedostoon.
- int [tasapainoitaPuu](#) (PUU *pAlku)
Palauttaa solmun tasapainokertoimen.
- [PUU * oikeaPuoli](#) (PUU *pAlku)
Oikean puolen kierto.
- [PUU * vasenPuoli](#) (PUU *pAlku)
Vasemman puolen kierto.
- int [puunPituus](#) (PUU *pAlku)
Hakee solmun korkeuden.
- int [suurempiLukuVertailu](#) (int iLuku1, int iLuku2)
Vertaa kahta lukua ja palauttaa suuremman.
- [PUU * poistaSolmu](#) (char *pNimi, [PUU](#) *pJuuriSolmu)

- *Lehtisolmun poistaminen.*
int **onkoLuku** (char *pNimi)
Testaa, onko käyttäjän antama arvo nimi vai numeroarvo.
- int **nimenArvo** (char *pNimi, **PUU** *pJuuriSolmu)
Etsii nimen perusteella sen arvon.
- **PUU** * **etsiPienin** (**PUU** *pUusiSolmu)
Etsii pienimmän solmun arvon binääripuusta.
- **PUU** * **paivitaPuu** (**PUU** *pJuuriSolmu)
Tasapainotetaan puu poiston jälkeen.

4.1.1 Function Documentation

4.1.1.1 etsiPienin()

```
PUU* etsiPienin (
    PUU * pUusiSolmu )
```

Parameters

<i>pUusiSolmu</i>	Osoitin kasiteltavaan solmuun.
-------------------	--------------------------------

Returns

PUU* Palauttaa osoittimen puun pienimpaan solmuun.

Definition at line 447 of file binaaripuu.c.

```
447     {
448     PUU *pNykyinenSolmu = pUusiSolmu;
449
450     while (pNykyinenSolmu->pVasen != NULL) {
451         pNykyinenSolmu = pNykyinenSolmu->pVasen;
452     }
453
454     return (pNykyinenSolmu);
455 }
```

4.1.1.2 kirjoitaBinaaripuu()

```
void kirjoitaBinaaripuu (
    char * pNimi,
    PUU * pAlku )
```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	

Definition at line 176 of file binaaripuu.c.

```

176                                     {
177     /*if (pAlku == NULL) { //Noora kommentoi pois 19.3.2026 klo. 13.08
178         printf("Puu on tyhjä, luo puurakenne ennen kirjoittamista.\n");
179         return;
180     }*/
181     if (pAlku != NULL) {
182         kirjoitaTiedostoon(pNimi, pAlku);
183         kirjoitaBinaaripuu(pNimi, pAlku->pVasen);
184         kirjoitaBinaaripuu(pNimi, pAlku->pOikea);
185     }
186     return;
187 }
```

4.1.1.3 kirjoitaTiedostoon()

```

void kirjoitaTiedostoon (
    char * pNimi,
    PUU * pAlku )
```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi
<i>pAlku</i>	Solmu, joka kirjoitetaan tiedostoon

Definition at line 195 of file binaaripuu.c.

```

195                                     {
196     FILE *Tiedosto = NULL;
197
198     if (pAlku == NULL) {
199         return;
200     }
201
202     /* Tiedoston avaaminen. */
203     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
204         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
205         exit(0);
206     }
207     /* Tiedostoon kirjoittaminen. */
208     fprintf(Tiedosto, "%s,%d\n", pAlku->aNimi, pAlku->iArvo);
209
210     /* Tiedoston sulkeminen. */
211     fclose(Tiedosto);
212     return;
213 }
```

4.1.1.4 lisaaSolmu()

```

PUU* lisaaSolmu (
    PUU * pAlku,
    char * pSolmu,
    int iArvo )
```

Tasapainottaa puun.

Parameters

<i>pAlku</i>	
<i>pSolmu</i>	Lisattava solmu.
<i>iArvo</i>	Solmun arvo.

Returns

PUU*

Definition at line 58 of file binaaripuu.c.

```

58                                     {
59     int iVertailu = 0;
60
61     // Varataan muistia, jos muisti tyhjä.
62     if (pAlku == NULL) {
63         return varaaMuistiaPuulle(pSolmu, iArvo);
64     }
65
66     iVertailu = strcmp(pSolmu, pAlku->aNimi);
67
68     // Solmujen lisääminen.
69     if (iArvo < pAlku->iArvo) {
70         pAlku->pVasen = lisaaSolmu(pAlku->pVasen, pSolmu, iArvo);
71     } else if (iArvo > pAlku->iArvo) {
72         pAlku->pOikea = lisaaSolmu(pAlku->pOikea, pSolmu, iArvo);
73     }
74
75     // Testataan, ovatko arvot samat.
76     } else if (iArvo == pAlku->iArvo) {
77         if (iVertailu < 0) {
78             pAlku->pVasen = lisaaSolmu(pAlku->pVasen, pSolmu, iArvo);
79         } else if (iVertailu > 0) {
80             pAlku->pOikea = lisaaSolmu(pAlku->pOikea, pSolmu, iArvo);
81         }
82     }
83
84     // Selvitetaan paikka, johon solmu lisataan. Tasapainotetaan puu.
85     pAlku->iPituus = 1 + suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea));
86     int tasapaino = tasapainoitaPuu(pAlku);
87
88     // vasen vasen tasapainotus
89     if ((tasapaino > 1) && (iArvo < pAlku->pVasen->iArvo)) {
90         return (oikeaPuoli(pAlku));
91     }
92
93     // vasen oikea tasapainotus
94     if ((tasapaino > 1) && (iArvo > pAlku->pVasen->iArvo)) {
95         pAlku->pVasen = vasenPuoli(pAlku->pVasen);
96         return (oikeaPuoli(pAlku));
97     }
98
99     // oikea vasen tasapainotus
100    if ((tasapaino < -1) && (iArvo < pAlku->pOikea->iArvo)) {
101        pAlku->pOikea = oikeaPuoli(pAlku->pOikea);
102        return (vasenPuoli(pAlku));
103    }
104
105    // oikea oikea tasapainotus
106    if ((tasapaino < -1) && (iArvo > pAlku->pOikea->iArvo)) {
107        return (vasenPuoli(pAlku));
108    }
109    return (pAlku);
110 }

```

4.1.1.5 luoPuu()

```

PUU* luoPuu (
    char * pNimi,
    PUU * pJuuriSolmu )

```

Pilkotaan luetut rivit.

Parameters

<i>pNimi</i>	Luettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Puun juurisolmu.

Returns

PUU* Palauttaa juuriSolmun.

Definition at line 119 of file binaaripuu.c.

```

119                                     {
120     FILE *Tiedosto = NULL;
121     char aRivi[LEN] = "";
122     int iOtsikko = 0;
123     char *p1 = NULL, *p2 = NULL;
124     int iArvo = 0;
125
126     // Tiedoston avaus
127     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
128         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
129         exit(0);
130     }
131
132     // Tiedoston lukeminen
133     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
134         aRivi[strlen(aRivi) - 1] = '\0';
135
136         // Ohitetaan otsikko
137         if (iOtsikko == 0) {
138             iOtsikko++;
139             continue;
140         }
141
142         // Pilkkotaan rivit
143         if ((p1 = strtok(aRivi, ";")) == NULL) {
144             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
145             exit(0);
146         }
147
148         if ((p2 = strtok(NULL, ";")) == NULL) {
149             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
150             exit(0);
151         }
152
153         iArvo = atoi(p2);
154
155         // Maaritetaan juuri solmu jos sitä ei ole määritetty.
156         if (pJuuriSolmu == NULL) {
157             pJuuriSolmu = lisaaSolmu(pJuuriSolmu, p1, iArvo);
158             iOtsikko++;
159             continue;
160         }
161
162         pJuuriSolmu = lisaaSolmu(pJuuriSolmu, p1, iArvo);
163     }
164
165     // Suljetaan tiedosto
166     fclose(Tiedosto);
167     return (pJuuriSolmu);
168 }
```

4.1.1.6 nimenArvo()

```

int nimenArvo (
    char * pNimi,
    PUU * pJuuriSolmu )
```

Parameters

<i>pNimi</i>	Poistettava nimi merkkijonona.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan solmuun.

Returns

int Palauttaa 0 tai 1, riippuen siitä, löytyykö nimen nimen pohjalta arvoa.

Definition at line 422 of file binaaripuu.c.

```

422                                     {
423     if (pJuuriSolmu == NULL) {
424         return (0);
425     }
426
427     if (strcmp(pJuuriSolmu->aNimi, pNimi) == 0) {
428         int iArvo = pJuuriSolmu->iArvo;
429         return (iArvo);
430     }
431
432     int iArvo = nimenArvo(pNimi, pJuuriSolmu->pVasen);
433     if (iArvo != 0) {
434         return (iArvo);
435     } else {
436         int iArvo = nimenArvo(pNimi, pJuuriSolmu->pOikea);
437         return (iArvo);
438     }
439 }

```

4.1.1.7 oikeaPuoli()

```

PUU* oikeaPuoli (
    PUU * pAlku )

```

Puu järjestetään, jotta se on tasapainossa.

Parameters

<i>pAlku</i>	
--------------	--

Returns

PUU* Palauttaa solmun.

Definition at line 237 of file binaaripuu.c.

```

237                                     {
238     // Tarkistetaan, onko pAlku tai pAlku oikea puoli NULL
239     if ((pAlku->pVasen == NULL) || (pAlku == NULL)) {
240         return (pAlku);
241     }
242
243     // Muuttujien ja vakioiden alustaminen
244     PUU *pMuuttujal = pAlku->pVasen;
245     PUU *pMuuttuja2 = pMuuttujal->pOikea;
246
247     // Sijoitetaan arvoja
248     pMuuttujal->pOikea = pAlku;
249     pAlku->pVasen = pMuuttuja2;
250
251     // Verrataan ja sijoitetaan suuremmat korkeudet
252     pAlku->iPituus = suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea)) + 1;
253     pMuuttujal->iPituus =
254         suurempiLukuVertailu(puunPituus(pMuuttujal->pVasen), puunPituus(pMuuttujal->pOikea)) + 1;
255
256     return pMuuttujal;
257 }

```

4.1.1.8 onkoLuku()

```

int onkoLuku (
    char * pNimi )

```

Ilmoittaa, onko käyttäjän antama numero vai ei.

Parameters

<i>pNimi</i>	Poistettava nimi tai numeroarvo merkkijonona.
--------------	---

Returns

int Palauttaa 0 tai 1, riippuen siitä onko käyttäjän syöte numero.

Definition at line 401 of file binaaripuu.c.

```
401     {
402         int tosi = 0;
403         char *ptr = pNimi;
404
405         while (*ptr) {
406             if (!isdigit(*ptr)) {
407                 return tosi;
408             }
409             ptr++;
410         }
411         tosi = 1;
412         return (tos);
413     }
```

4.1.1.9 paivitaPuu()

```
PUU* paivitaPuu (
    PUU * pJuuriSolmu )
```

Parameters

<i>pJuuriSolmu</i>	Osoitin kasiteltavaan solmuun.
--------------------	--------------------------------

Returns

PUU* palauttaa tasapainotetun solmun.

Definition at line 463 of file binaaripuu.c.

```
463     {
464         // Puun korkeuden päivittäminen.
465         if (pJuuriSolmu == NULL) {
466             return (pJuuriSolmu);
467         }
468
469         int iLuku1 = puunPituus(pJuuriSolmu->pVasen);
470         int iLuku2 = puunPituus(pJuuriSolmu->pOikea);
471         pJuuriSolmu->iPituus = suurempiLukuVertailu(iLuku1, iLuku2) + 1;
472
473         // Puun tasapanottaminen.
474         int iTasapaino = tasapainoitaPuu(pJuuriSolmu);
475         // Vasen-vasen tapaus.
476         if (iTasapaino > 1 && tasapainoitaPuu(pJuuriSolmu->pVasen) >= 0) {
477             return oikeaPuoli(pJuuriSolmu);
478         }
479
480         // Oikea-oikea tapaus.
481         if (iTasapaino < -1 && tasapainoitaPuu(pJuuriSolmu->pOikea) <= 0) {
482             return vasenPuoli(pJuuriSolmu);
483         }
484
485         // Vasen-oikea tapaus.
486         if (iTasapaino > 1 && tasapainoitaPuu(pJuuriSolmu->pVasen) < 0) {
487             pJuuriSolmu->pVasen = vasenPuoli(pJuuriSolmu->pVasen);
488             return oikeaPuoli(pJuuriSolmu);
489         }
```

```

489     }
490
491     // Oikea-vasen tapaus.
492     if (iTasapaino < -1 && tasapainoitaPuu(pJuuriSolmu->pOikea) > 0) {
493         pJuuriSolmu->pOikea = oikeaPuoli(pJuuriSolmu->pOikea);
494         return vasenPuoli(pJuuriSolmu);
495     }
496
497     return (pJuuriSolmu);
498 }

```

4.1.1.10 poistaSolmu()

```

PUU* poistaSolmu (
    char * pNimi,
    PUU * pJuuriSolmu )

```

Poistaa lehtisolmun joko numeroarvon tai nimen perusteella.

Parameters

<i>pNimi</i>	Poistettava nimi tai numeroarvo merkkijonona.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan puun solmuun.
<i>iArvo</i>	Haettava numeroarvo.

Returns

PUU* Uusi osoitin puun solmuun, NULL jos puu tyhjeni.

Definition at line 334 of file binaaripuu.c.

```

334     {
335         int iArvo = 0;
336         PUU *pUusiSolmu = NULL;
337
338         if (pJuuriSolmu == NULL) {
339             printf("Solmua ei löytynyt.\n");
340             return (pJuuriSolmu);
341         }
342
343         // Testataan, onko syöte nimi vai lukuarvo, ja määritetään lukuarvo puun läpikäymiseksi.
344         if (onkoLuku(pNimi)) {
345             iArvo = atoi(pNimi);
346         } else {
347             iArvo = nimenArvo(pNimi, pJuuriSolmu);
348
349             if (iArvo == 0) {
350                 printf("Solmua ei löytynyt tällä nimellä.\n");
351                 return (pJuuriSolmu);
352             }
353         }
354
355         // Lehtisolmun poistaminen.
356         if (iArvo < pJuuriSolmu->iArvo) {
357             pJuuriSolmu->pVasen = poistaSolmu(pNimi, pJuuriSolmu->pVasen);
358         } else if (iArvo > pJuuriSolmu->iArvo) {
359             pJuuriSolmu->pOikea = poistaSolmu(pNimi, pJuuriSolmu->pOikea);
360         } else {
361
362             if (pJuuriSolmu->pVasen == NULL && pJuuriSolmu->pOikea == NULL) {
363                 free(pJuuriSolmu);
364                 pJuuriSolmu = NULL;
365                 printf("Solmu poistettu.\n");
366                 return (pJuuriSolmu);
367             } else if (pJuuriSolmu->pVasen == NULL || pJuuriSolmu->pOikea == NULL) {
368                 if (pJuuriSolmu->pVasen != NULL) {
369                     pUusiSolmu = pJuuriSolmu->pVasen;
370                 } else {

```

```

372         pUusiSolmu = pJuuriSolmu->pOikea;
373     }
374     free(pJuuriSolmu);
375     pJuuriSolmu = NULL;
376     printf("Solmu poistettu.\n");
377     return (pUusiSolmu);
378 }
379 } else {
380     pUusiSolmu = etsiPienin(pJuuriSolmu->pOikea);
381     pJuuriSolmu->iArvo = pUusiSolmu->iArvo;
382     strcpy(pJuuriSolmu->aNimi, pUusiSolmu->aNimi);
383     pJuuriSolmu->pOikea = poistaSolmu(pUusiSolmu->aNimi, pJuuriSolmu->pOikea);
384     return (pJuuriSolmu);
385 }
386 }
387
388 pJuuriSolmu = paivitaPuu(pJuuriSolmu);
389 return (pJuuriSolmu);
390 }

```

4.1.1.11 puunPituus()

```

int puunPituus (
    PUU * pAlku )

```

Parameters

<i>pAlku</i>	Kohta, josta korkeus haetaan
--------------	------------------------------

Returns

int Palauttaa solmun korkeuden

Definition at line 293 of file binaaripuu.c.

```

293     {
294         // Tarkistetaan, onko Solmu Null
295         if (pAlku == NULL) {
296             return (0);
297         }
298
299         // Palauttaa solmun korkeuden
300         return (pAlku->iPituus);
301     }

```

4.1.1.12 suurempiLukuVertailu()

```

int suurempiLukuVertailu (
    int iLuku1,
    int iLuku2 )

```

Parameters

<i>iLuku1</i>	Ensimmäinen verrattava kokonaisluku.
<i>iLuku2</i>	Toinen verrattava kokonaisluku.

Returns

int Palauttaa suuremman luvun.

Definition at line 310 of file binaaripuu.c.

```
310                                     {
311     int iPalautus = 0;
312
313     // Verrataan kumpi luvuista on suurempi.
314     if (iLuku1 > iLuku2) {
315         iPalautus = iLuku1;
316     } else {
317         iPalautus = iLuku2;
318     }
319
320     // Palauttaa suuremman luvun.
321     return (iPalautus);
322 }
```

4.1.1.13 tasapainoitaPuu()

```
int tasapainoitaPuu (
    PUU * pAlku )
```

Parameters

<i>pAlku</i>	Solmu, josta tasapainokerroin halutaan
--------------	--

Returns

int

Definition at line 221 of file binaaripuu.c.

```
221                                     {
222     // Tarkistetaan, onko Null
223     if (pAlku == NULL) {
224         return (0);
225     }
226
227     // Palautetaan tasapainokerroin
228     return (puunPituus(pAlku->pVasen) - puunPituus(pAlku->pOikea));
229 }
```

4.1.1.14 vapautaMuistiPuu()

```
PUU* vapautaMuistiPuu (
    PUU * pJuuriSolmu )
```

Parameters

<i>pJuuriSolmu</i>	Puun juurisolmu
--------------------	-----------------

Returns

PUU*

Definition at line 39 of file binaaripuu.c.

```

39                                     {
40     // Muistin vapauttaminen
41     if (pJuuriSolmu != NULL) {
42         vapautaMuistiPuu(pJuuriSolmu->pVasen);
43         vapautaMuistiPuu(pJuuriSolmu->pOikea);
44         free(pJuuriSolmu);
45     }
46     pJuuriSolmu = NULL;
47     return (pJuuriSolmu);
48 }

```

4.1.1.15 varaaMuistiaPuulle()

```

PUU* varaaMuistiaPuulle (
    char * pSolmu,
    int iArvo )

```

Parameters

<i>pSolmu</i>	Solmu, jolle muistia varataan
<i>iArvo</i>	Arvo, jolle muistia varataan

Returns

PUU*

Definition at line 16 of file binaaripuu.c.

```

16                                     {
17     PUU *pUusi = NULL;
18
19     // Muistin varaaminen
20     if ((pUusi = (PUU *)malloc(sizeof(PUU))) == NULL) {
21         perror("Muistin varaus epäonnistui, lopetetaan");
22         exit(0);
23     }
24
25     strcpy(pUusi->aNimi, pSolmu);
26     pUusi->iArvo = iArvo;
27     pUusi->iPituus = 1;
28     pUusi->pVasen = NULL;
29     pUusi->pOikea = NULL;
30     return (pUusi);
31 }

```

4.1.1.16 vasenPuoli()

```

PUU* vasenPuoli (
    PUU * pAlku )

```

Puu järjestetään, jotta se on tasapainossa.

Parameters

<i>pAlku</i>	
--------------	--

Returns

PUU* Palauttaa solmun.

Definition at line 265 of file binaaripuu.c.

```

265     {
266         // Tarkistetaan, onko pAlku tai pAlku oikea puoli NULL
267         if ((pAlku->pOikea == NULL) || (pAlku == NULL)) {
268             return (pAlku);
269         }
270
271         // Muuttujien ja vakioiden alustaminen
272         PUU *pMuuttuja1 = pAlku->pOikea;
273         PUU *pMuuttuja2 = pMuuttuja1->pVasen;
274
275         // Sijoitetetaan arvoja
276         pMuuttuja1->pVasen = pAlku;
277         pAlku->pOikea = pMuuttuja2;
278
279         // Verrataan ja sijoitetaan suuremmat korkeudet
280         pAlku->iPituus = suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea)) + 1;
281         pMuuttuja1->iPituus =
282             suurempiLukuVertailu(puunPituus(pMuuttuja1->pVasen), puunPituus(pMuuttuja1->pOikea)) + 1;
283
284         return pMuuttuja1;
285     }

```

4.2 binaaripuu.h File Reference

```
#include "yleiset.h"
```

Classes

- struct [puu](#)
- struct [RBSolmu](#)

Typedefs

- typedef struct [puu](#) PUU
- typedef struct [RBSolmu](#) RBSOLMU

Functions

- [PUU * varaaMuistiaPuulle](#) (char *pSolmu, int iArvo)
Muistin varaaminen puu structin alkioille.
- [PUU * vapautaMuistiPuu](#) (PUU *pJuuriSolmu)
Muistin vapauttaminen puu structin alkioista Juurisolmun ja kaikkien sen lapsie solmujen muisti vapautetaan.
- [PUU * lisaaSolmu](#) (PUU *pAlku, char *pSolmu, int iArvo)
Lisaa solmun puuhun.
- [PUU * luoPuu](#) (char *pNimi, [PUU](#) *pJuuriSolmu)

- Luetaan tiedosto ja luodaan puu rakenne.*
- int `tasapainoitaPuu` (`PUU *pAlku`)
- Palauttaa solmun tasapainokertoimen.*
- `PUU * oikeaPuoli` (`PUU *pAlku`)
- Oikean puolen kierto.*
- `PUU * vasenPuoli` (`PUU *pAlku`)
- Vasemman puolen kierto.*
- int `puunPituus` (`PUU *pAlku`)
- Hakee solmun korkeuden.*
- int `suurempiLukuVertailu` (int `iLuku1`, int `iLuku2`)
- Vertaa kahta lukua ja palauttaa suuremman.*
- `PUU * poistaSolmu` (char `*pNimi`, `PUU *pJuuriSolmu`)
- Lehtisolmun poistaminen.*
- int `onkoLuku` (char `*pNimi`)
- Testaa, onko käyttäjän antama arvo nimi vai numeroarvo.*
- int `nimenArvo` (char `*pNimi`, `PUU *pJuuriSolmu`)
- Etsii nimen perusteella sen arvon.*
- `PUU * etsiPienin` (`PUU *pUusiSolmu`)
- Etsii pienimmän solmun arvon binääripuusta.*
- `PUU * paivitaPuu` (`PUU *pJuuriSolmu`)
- Tasapainotetaan puu poiston jälkeen.*
- void `kirjoitaBinaaripuu` (char `*pNimi`, `PUU *pAlku`)
- Tasapainoitettun binaaripuun kirjoittaminen tiedostoon.*
- void `kirjoitaTiedostoon` (char `*pKirjoitaTiedostoNimi`, `PUU *pAlku`)
- Kirjoittaa binaaripuun tiedostoon.*
- `RBSOLMU * varaaMuistiaRB` (char `*pNimi`, int `iArvo`)
- Punamustapuu, ehkä oma header olisi kätevämpi, kun omat aliohjelmat oman structin takia.*
- `RBSOLMU * vapautaMuistiRB` (`RBSOLMU *pJuuriSolmu`)
- `RBSOLMU * lisaaRBSolmu` (`RBSOLMU *pJuuriSolmu`, `RBSOLMU *pUusi`)
- `RBSOLMU * luoRBPuu` (`RBSOLMU *pJuuriSolmu`, char `*pNimi`)
- void `kierraVasemmalle` (`RBSOLMU **pJuuriSolmu`, `RBSOLMU *pSolmu`)
- void `kierraOikealle` (`RBSOLMU **pJuuriSolmu`, `RBSOLMU *pSolmu`)
- void `korjaaLisays` (`RBSOLMU **pJuuriSolmu`, `RBSOLMU *pUusi`)
- void `kirjoitaRBTiedostoon` (char `*pNimi`, `RBSOLMU *pJuuriSolmu`)
- void `kirjoitaRB` (char `*pNimi`, `RBSOLMU *pJuuriSolmu`)

4.2.1 Typedef Documentation

4.2.1.1 PUU

```
typedef struct puu PUU
```

4.2.1.2 RBSOLMU

```
typedef struct RBSolmu RBSOLMU
```

4.2.2 Function Documentation

4.2.2.1 etsiPienin()

```
PUU* etsiPienin (
    PUU * pUusiSolmu )
```

Parameters

<i>pUusiSolmu</i>	Osoitin kasiteltavaan solmuun.
-------------------	--------------------------------

Returns

PUU* Palauttaa osoittimen puun pienimpaan solmuun.

Definition at line 447 of file binaaripuu.c.

```
447     {
448         PUU *pNykyinenSolmu = pUusiSolmu;
449
450         while (pNykyinenSolmu->pVasen != NULL) {
451             pNykyinenSolmu = pNykyinenSolmu->pVasen;
452         }
453
454         return (pNykyinenSolmu);
455     }
```

4.2.2.2 kierraOikealle()

```
void kierraOikealle (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pSolmu )
```

Definition at line 130 of file punamustapuu.c.

```
130     {
131         RBSOLMU *pVasenLapsi = pSolmu->pVasen;
132         pSolmu->pVasen = pVasenLapsi->pOikea;
133
134         if (pSolmu->pVasen != NULL)
135             pSolmu->pVasen->pVanhempi = pSolmu;
136
137         pVasenLapsi->pVanhempi = pSolmu->pVanhempi;
138
139         if (pSolmu->pVanhempi == NULL)
140             *pJuurisolmu = pVasenLapsi;
141         else if (pSolmu == pSolmu->pVanhempi->pVasen)
142             pSolmu->pVanhempi->pVasen = pVasenLapsi;
143         else
144             pSolmu->pVanhempi->pOikea = pVasenLapsi;
145
146         pVasenLapsi->pOikea = pSolmu;
147         pSolmu->pVanhempi = pVasenLapsi;
148     }
```


4.2.2.3 kierraVasemmalle()

```
void kierraVasemmalle (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pSolmu )
```

Definition at line 109 of file punamustapuu.c.

```
109
110     RBSOLMU *pOikeaLapsi = pSolmu->pOikea;
111     pSolmu->pOikea = pOikeaLapsi->pVasen;
112
113     if (pSolmu->pOikea != NULL)
114         pSolmu->pOikea->pVanhempi = pSolmu;
115
116     // Solmun oikea lapsi nousee ylös puussa
117     pOikeaLapsi->pVanhempi = pSolmu->pVanhempi;
118
119     if (pSolmu->pVanhempi == NULL)
120         *pJuurisolmu = pOikeaLapsi;
121     else if (pSolmu == pSolmu->pVanhempi->pVasen)
122         pSolmu->pVanhempi->pVasen = pOikeaLapsi;
123     else
124         pSolmu->pVanhempi->pOikea = pOikeaLapsi;
125
126     pOikeaLapsi->pVasen = pSolmu;
127     pSolmu->pVanhempi = pOikeaLapsi;
128 }
```

4.2.2.4 kirjoitaBinaaripuu()

```
void kirjoitaBinaaripuu (
    char * pNimi,
    PUU * pAlku )
```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	

Definition at line 176 of file binaaripuu.c.

```
176
177     /*if (pAlku == NULL) { //Noora kommentoi pois 19.3.2026 klo. 13.08
178         printf("Puu on tyhjä, luo puurakenne ennen kirjoittamista.\n");
179         return;
180     }*/
181     if (pAlku != NULL) {
182         kirjoitaTiedostoon(pNimi, pAlku);
183         kirjoitaBinaaripuu(pNimi, pAlku->pVasen);
184         kirjoitaBinaaripuu(pNimi, pAlku->pOikea);
185     }
186     return;
187 }
```

4.2.2.5 kirjoitaRB()

```
void kirjoitaRB (
    char * pNimi,
    RBSOLMU * pJuurisolmu )
```

Definition at line 222 of file punamustapuu.c.

```

222                                     {
223     /*if (pJuurisolmu == NULL) {
224         printf("Puu on tyhjä, luo puurakenne ennen kirjoittamista.\n");
225         return;
226     }*/
227     if (pJuurisolmu != NULL) {
228         kirjoitaRBTiedostoon(pNimi, pJuurisolmu);
229         kirjoitaRB(pNimi, pJuurisolmu->pVasen);
230         kirjoitaRB(pNimi, pJuurisolmu->pOikea);
231     }
232     return;
233 }
```

4.2.2.6 kirjoitaRBTiedostoon()

```

void kirjoitaRBTiedostoon (
    char * pNimi,
    RBSOLMU * pJuurisolmu )
```

Definition at line 201 of file punamustapuu.c.

```

201                                     {
202     FILE *Tiedosto = NULL;
203
204     if (pJuurisolmu == NULL) {
205         return;
206     }
207
208     /* Tiedoston avaaminen. */
209     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
210         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
211         exit(0);
212     }
213     /* Tiedostoon kirjoittaminen. */
214     fprintf(Tiedosto, "%s,%d\n", pJuurisolmu->aNimi, pJuurisolmu->iArvo);
215
216     /* Tiedoston sulkeminen. */
217     fclose(Tiedosto);
218     printf("Tiedosto '%s' kirjoitettu.\n", pNimi);
219     return;
220 }
```

4.2.2.7 kirjoitaTiedostoon()

```

void kirjoitaTiedostoon (
    char * pNimi,
    PUU * pAlku )
```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi
<i>pAlku</i>	Solmu, joka kirjoitetaan tiedostoon

Definition at line 195 of file binaaripuu.c.

```

195                                     {
196     FILE *Tiedosto = NULL;
197
198     if (pAlku == NULL) {
199         return;
200     }
201
202     /* Tiedoston avaaminen. */
```

```

203     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
204         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
205         exit(0);
206     }
207     /* Tiedostoon kirjoittaminen. */
208     fprintf(Tiedosto, "%s,%d\n", pAlku->aNimi, pAlku->iArvo);
209
210     /* Tiedoston sulkeminen. */
211     fclose(Tiedosto);
212     return;
213 }

```

4.2.2.8 korjaaLisays()

```

void korjaaLisays (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pUusi )

```

Definition at line 150 of file punamustapuu.c.

```

150
151     // Ilman p-alkua, koska pVanhempi->pVanhempi oli sekava
152     RBSOLMU *vanhempi = NULL;
153     RBSOLMU *seta = NULL;
154     RBSOLMU *isoVanhempi = NULL;
155
156     while ((pUusi != *pJuurisolmu) && (pUusi->pVanhempi->iVariBitti == 0)) {
157         vanhempi = pUusi->pVanhempi;
158         isoVanhempi = vanhempi->pVanhempi;
159
160         if (vanhempi == isoVanhempi->pVasen) {
161             seta = isoVanhempi->pOikea;
162             if (seta != NULL && seta->iVariBitti == 0) {
163                 vanhempi->iVariBitti = 1;
164                 seta->iVariBitti = 1;
165                 isoVanhempi->iVariBitti = 0;
166                 pUusi = isoVanhempi;
167             } else {
168                 if (pUusi == vanhempi->pOikea) {
169                     pUusi = vanhempi;
170                     kierraVasemmalle(pJuurisolmu, pUusi);
171                     vanhempi = pUusi->pVanhempi;
172                 }
173                 vanhempi->iVariBitti = 1;
174                 isoVanhempi->iVariBitti = 0;
175                 kierraOikealle(pJuurisolmu, isoVanhempi);
176             }
177
178         } else {
179             seta = isoVanhempi->pVasen;
180             if (seta != NULL && seta->iVariBitti == 0) {
181                 vanhempi->iVariBitti = 1;
182                 seta->iVariBitti = 1;
183                 isoVanhempi->iVariBitti = 0;
184                 pUusi = isoVanhempi;
185             } else {
186                 if (pUusi == vanhempi->pVasen) {
187                     pUusi = vanhempi;
188                     kierraOikealle(pJuurisolmu, pUusi);
189                     vanhempi = pUusi->pVanhempi;
190                 }
191                 vanhempi->iVariBitti = 1;
192                 isoVanhempi->iVariBitti = 0;
193                 kierraVasemmalle(pJuurisolmu, isoVanhempi);
194             }
195         }
196     }
197     (*pJuurisolmu)->iVariBitti = 1;
198 }

```

4.2.2.9 lisaaRBSolmu()

```
RBSOLMU* lisaaRBSolmu (
    RBSOLMU * pJuurisolmu,
    RBSOLMU * pUusi )
```

Katsotaan, ovatko arvot samat. Laitetaan aakkosten perusteella vasemmalle tai oikealle.

Definition at line 38 of file punamustapuu.c.

```
38                                     {
39     int iVertailu = 0;
40
41     // Jos puu on tyhjä, palautetaan uusi RBSolmu.
42     if (pJuurisolmu == NULL) {
43         return pUusi;
44     }
45
46     iVertailu = strcmp(pUusi->aNimi, pJuurisolmu->aNimi);
47
48     // Solmujen lisääminen rekursiivisesti
49     if (pUusi->iArvo < pJuurisolmu->iArvo) {
50         pJuurisolmu->pVasen = lisaaRBSolmu(pJuurisolmu->pVasen, pUusi);
51         pJuurisolmu->pVasen->pVanhempi = pJuurisolmu;
52     } else if (pUusi->iArvo > pJuurisolmu->iArvo) {
53         pJuurisolmu->pOikea = lisaaRBSolmu(pJuurisolmu->pOikea, pUusi);
54         pJuurisolmu->pOikea->pVanhempi = pJuurisolmu;
55     } else if (pUusi->iArvo == pJuurisolmu->iArvo) {
56         if (iVertailu < 0) {
57             pJuurisolmu->pVasen = lisaaRBSolmu(pJuurisolmu->pVasen, pUusi);
58             pJuurisolmu->pVasen->pVanhempi = pJuurisolmu;
59         } else if (iVertailu > 0) {
60             pJuurisolmu->pOikea = lisaaRBSolmu(pJuurisolmu->pOikea, pUusi);
61             pJuurisolmu->pOikea->pVanhempi = pJuurisolmu;
62         }
63     }
64     return (pJuurisolmu);
65 }
66
67 }
```

4.2.2.10 lisaaSolmu()

```
PUU* lisaaSolmu (
    PUU * pAlku,
    char * pSolmu,
    int iArvo )
```

Tasapainottaa puun.

Parameters

<i>pAlku</i>	
<i>pSolmu</i>	Lisattava solmu.
<i>iArvo</i>	Solmun arvo.

Returns

PUU*

Definition at line 58 of file binaaripuu.c.

```
58                                     {
59     int iVertailu = 0;
60
61     // Varataan muistia, jos muisti tyhjä.
```

```

62     if (pAlku == NULL) {
63         return varaaMuistiaPuulle(pSolmu, iArvo);
64     }
65
66     iVertailu = strcmp(pSolmu, pAlku->aNimi);
67
68     // Solmujen lisääminen.
69     if (iArvo < pAlku->iArvo) {
70         pAlku->pVasen = lisaaSolmu(pAlku->pVasen, pSolmu, iArvo);
71     } else if (iArvo > pAlku->iArvo) {
72         pAlku->pOikea = lisaaSolmu(pAlku->pOikea, pSolmu, iArvo);
73
74         // Testataan, ovatko arvot samat.
75     } else if (iArvo == pAlku->iArvo) {
76         if (iVertailu < 0) {
77             pAlku->pVasen = lisaaSolmu(pAlku->pVasen, pSolmu, iArvo);
78         } else if (iVertailu > 0) {
79             pAlku->pOikea = lisaaSolmu(pAlku->pOikea, pSolmu, iArvo);
80         }
81     }
82
83     // Selvitetaan paikka, johon solmu lisataan. Tasapainotetaan puu.
84     pAlku->iPituus = 1 + suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea));
85     int tasapaino = tasapainoitaPuu(pAlku);
86
87     // vasen vasen tasapainotus
88     if ((tasapaino > 1) && (iArvo < pAlku->pVasen->iArvo)) {
89         return (oikeaPuoli(pAlku));
90     }
91
92     // vasen oikea tasapainotus
93     if ((tasapaino > 1) && (iArvo > pAlku->pVasen->iArvo)) {
94         pAlku->pVasen = vasenPuoli(pAlku->pVasen);
95         return (oikeaPuoli(pAlku));
96     }
97
98     // oikea vasen tasapainotus
99     if ((tasapaino < -1) && (iArvo < pAlku->pOikea->iArvo)) {
100         pAlku->pOikea = oikeaPuoli(pAlku->pOikea);
101         return (vasenPuoli(pAlku));
102     }
103
104     // oikea oikea tasapainotus
105     if ((tasapaino < -1) && (iArvo > pAlku->pOikea->iArvo)) {
106         return (vasenPuoli(pAlku));
107     }
108
109     return (pAlku);
110 }

```

4.2.2.11 luoPuu()

```

PUU* luoPuu (
    char * pNimi,
    PUU * pJuuriSolmu )

```

Pilkotaan luetut rivit.

Parameters

<i>pNimi</i>	Luettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Puun juurisolmu.

Returns

PUU* Palauttaa juuriSolmun.

Definition at line 119 of file binaaripuu.c.

```

119                                     {
120     FILE *Tiedosto = NULL;
121     char aRivi[LEN] = "";
122     int iOtsikko = 0;
123     char *p1 = NULL, *p2 = NULL;
124     int iArvo = 0;
125
126     // Tiedoston avaus
127     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
128         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
129         exit(0);
130     }
131
132     // Tiedoston lukeminen
133     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
134         aRivi[strlen(aRivi) - 1] = '\0';
135
136         // Ohitetaan otsikko
137         if (iOtsikko == 0) {
138             iOtsikko++;
139             continue;
140         }
141
142         // Pilkotaan rivit
143         if ((p1 = strtok(aRivi, ";")) == NULL) {
144             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
145             exit(0);
146         }
147
148         if ((p2 = strtok(NULL, ";")) == NULL) {
149             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
150             exit(0);
151         }
152
153         iArvo = atoi(p2);
154
155         // Maaritetaan juuri solmu jos sitä ei ole määriteltty.
156         if (pJuuriSolmu == NULL) {
157             pJuuriSolmu = lisaaSolmu(pJuuriSolmu, p1, iArvo);
158             iOtsikko++;
159             continue;
160         }
161
162         pJuuriSolmu = lisaaSolmu(pJuuriSolmu, p1, iArvo);
163     }
164
165     // Suljetaan tiedosto
166     fclose(Tiedosto);
167     return (pJuuriSolmu);
168 }

```

4.2.2.12 `luoRBPuu()`

```

RBSOLMU* luoRBPuu (
    RBSOLMU * pJuurisolmu,
    char * pNimi )

```

Definition at line 69 of file `punamustapuu.c`.

```

69                                     {
70     FILE *Tiedosto = NULL;
71     char aRivi[LEN] = "";
72     int iOtsikko = 0;
73     char *p1 = NULL, *p2 = NULL;
74     int iArvo = 0;
75
76     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
77         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
78         exit(0);
79     }
80
81     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
82         aRivi[strlen(aRivi) - 1] = '\0';
83         if (iOtsikko == 0) {
84             iOtsikko++;
85             continue;
86         }
87
88         if ((p1 = strtok(aRivi, ";")) == NULL) {

```

```

89         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
90         exit(0);
91     }
92
93     if ((p2 = strtok(NULL, ";")) == NULL) {
94         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
95         exit(0);
96     }
97
98     iArvo = atoi(p2);
99
100     RBSOLMU *pUusi = varaaMuistiaRB(pl, iArvo);
101     pJuuriSolmu = lisaaRBSolmu(pJuuriSolmu, pUusi);
102     korjaaLisays(&pJuuriSolmu, pUusi);
103 }
104
105 fclose(Tiedosto);
106 return (pJuuriSolmu);
107 }

```

4.2.2.13 nimenArvo()

```

int nimenArvo (
    char * pNimi,
    PUU * pJuuriSolmu )

```

Parameters

<i>pNimi</i>	Poistettava nimi merkkijonona.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan solmuun.

Returns

int Palauttaa 0 tai 1, riippuen siitä, löytyykö nimen nimen pohjalta arvoa.

Definition at line 422 of file binaaripuu.c.

```

422                                     {
423     if (pJuuriSolmu == NULL) {
424         return (0);
425     }
426
427     if (strcmp(pJuuriSolmu->aNimi, pNimi) == 0) {
428         int iArvo = pJuuriSolmu->iArvo;
429         return (iArvo);
430     }
431
432     int iArvo = nimenArvo(pNimi, pJuuriSolmu->pVasen);
433     if (iArvo != 0) {
434         return (iArvo);
435     } else {
436         int iArvo = nimenArvo(pNimi, pJuuriSolmu->pOikea);
437         return (iArvo);
438     }
439 }

```

4.2.2.14 oikeaPuoli()

```

PUU* oikeaPuoli (
    PUU * pAlku )

```

Puu järjestetään, jotta se on tasapainossa.

Parameters

<i>pAlku</i>	
--------------	--

Returns

PUU* Palauttaa solmun.

Definition at line 237 of file binaaripuu.c.

```

237     {
238         // Tarkistetaan, onko pAlku tai pAlku oikea puoli NULL
239         if ((pAlku->pVasen == NULL) || (pAlku == NULL)) {
240             return (pAlku);
241         }
242
243         // Muuttujien ja vakioiden alustaminen
244         PUU *pMuuttujal = pAlku->pVasen;
245         PUU *pMuuttuja2 = pMuuttujal->pOikea;
246
247         // Sijoitetetaan arvoja
248         pMuuttujal->pOikea = pAlku;
249         pAlku->pVasen = pMuuttuja2;
250
251         // Verrataan ja sijoitetaan suuremmat korkeudet
252         pAlku->iPituus = suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea)) + 1;
253         pMuuttujal->iPituus =
254             suurempiLukuVertailu(puunPituus(pMuuttujal->pVasen), puunPituus(pMuuttujal->pOikea)) + 1;
255
256         return pMuuttujal;
257     }

```

4.2.2.15 onkoLuku()

```

int onkoLuku (
    char * pNimi )

```

Ilmoittaa, onko käyttäjän antama numero vai ei.

Parameters

<i>pNimi</i>	Poistettava nimi tai numeroarvo merkkijonona.
--------------	---

Returns

int Palauttaa 0 tai 1, riippuen siitä onko käyttäjän syöte numero.

Definition at line 401 of file binaaripuu.c.

```

401     {
402         int tosi = 0;
403         char *ptr = pNimi;
404
405         while (*ptr) {
406             if (!isdigit(*ptr)) {
407                 return tosi;
408             }
409             ptr++;
410         }
411         tosi = 1;
412         return (tos);
413     }

```


4.2.2.16 paivitaPuu()

```
PUU* paivitaPuu (
    PUU * pJuuriSolmu )
```

Parameters

<i>pJuuriSolmu</i>	Osoitin kasiteltavaan solmuun.
--------------------	--------------------------------

Returns

PUU* palauttaa tasapainotetun solmun.

Definition at line 463 of file binaaripuu.c.

```
463 {
464     // Puun korkeuden päivittäminen.
465     if (pJuuriSolmu == NULL) {
466         return (pJuuriSolmu);
467     }
468
469     int iLuku1 = puunPituus(pJuuriSolmu->pVasen);
470     int iLuku2 = puunPituus(pJuuriSolmu->pOikea);
471     pJuuriSolmu->iPituus = suurempiLukuVertailu(iLuku1, iLuku2) + 1;
472
473     // Puun tasapanottaminen.
474     int iTasapaino = tasapainoitaPuu(pJuuriSolmu);
475     // Vasen-vasen tapaus.
476     if (iTasapaino > 1 && tasapainoitaPuu(pJuuriSolmu->pVasen) >= 0) {
477         return oikeaPuoli(pJuuriSolmu);
478     }
479
480     // Oikea-oikea tapaus.
481     if (iTasapaino < -1 && tasapainoitaPuu(pJuuriSolmu->pOikea) <= 0) {
482         return vasenPuoli(pJuuriSolmu);
483     }
484
485     // Vasen-oikea tapaus.
486     if (iTasapaino > 1 && tasapainoitaPuu(pJuuriSolmu->pVasen) < 0) {
487         pJuuriSolmu->pVasen = vasenPuoli(pJuuriSolmu->pVasen);
488         return oikeaPuoli(pJuuriSolmu);
489     }
490
491     // Oikea-vasen tapaus.
492     if (iTasapaino < -1 && tasapainoitaPuu(pJuuriSolmu->pOikea) > 0) {
493         pJuuriSolmu->pOikea = oikeaPuoli(pJuuriSolmu->pOikea);
494         return vasenPuoli(pJuuriSolmu);
495     }
496
497     return (pJuuriSolmu);
498 }
```

4.2.2.17 poistaSolmu()

```
PUU* poistaSolmu (
    char * pNimi,
    PUU * pJuuriSolmu )
```

Poistaa lehtisolmun joko numeroarvon tai nimen perusteella.

Parameters

<i>pNimi</i>	Poistettava nimi tai numeroarvo merkkijonona.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan puun solmuun.
<i>iArvo</i>	Haettava numeroarvo.

Returns

PUU* Uusi osoitin puun solmuun, NULL jos puu tyhjeni.

Definition at line 334 of file binaaripuu.c.

```

334                                     {
335     int iArvo = 0;
336     PUU *pUusiSolmu = NULL;
337
338     if (pJuuriSolmu == NULL) {
339         printf("Solmua ei löytynyt.\n");
340         return (pJuuriSolmu);
341     }
342
343     // Testataan, onko syöte nimi vai lukuarvo, ja määritetään lukuarvo puun läpikäymiseksi.
344     if (onkoLuku(pNimi)) {
345         iArvo = atoi(pNimi);
346     } else {
347         iArvo = nimenArvo(pNimi, pJuuriSolmu);
348
349         if (iArvo == 0) {
350             printf("Solmua ei löytynyt tällä nimellä.\n");
351             return (pJuuriSolmu);
352         }
353     }
354
355     // Lehtisolmun poistaminen.
356     if (iArvo < pJuuriSolmu->iArvo) {
357         pJuuriSolmu->pVasen = poistaSolmu(pNimi, pJuuriSolmu->pVasen);
358     } else if (iArvo > pJuuriSolmu->iArvo) {
359         pJuuriSolmu->pOikea = poistaSolmu(pNimi, pJuuriSolmu->pOikea);
360     } else {
361
362         if (pJuuriSolmu->pVasen == NULL && pJuuriSolmu->pOikea == NULL) {
363             free(pJuuriSolmu);
364             pJuuriSolmu = NULL;
365             printf("Solmu poistettu.\n");
366             return (pJuuriSolmu);
367
368         } else if (pJuuriSolmu->pVasen == NULL || pJuuriSolmu->pOikea == NULL) {
369             if (pJuuriSolmu->pVasen != NULL) {
370                 pUusiSolmu = pJuuriSolmu->pVasen;
371             } else {
372                 pUusiSolmu = pJuuriSolmu->pOikea;
373             }
374             free(pJuuriSolmu);
375             pJuuriSolmu = NULL;
376             printf("Solmu poistettu.\n");
377             return (pUusiSolmu);
378
379         } else {
380             pUusiSolmu = etsiPienin(pJuuriSolmu->pOikea);
381             pJuuriSolmu->iArvo = pUusiSolmu->iArvo;
382             strcpy(pJuuriSolmu->aNimi, pUusiSolmu->aNimi);
383             pJuuriSolmu->pOikea = poistaSolmu(pUusiSolmu->aNimi, pJuuriSolmu->pOikea);
384             return (pJuuriSolmu);
385         }
386     }
387
388     pJuuriSolmu = paivitaPuu(pJuuriSolmu);
389     return (pJuuriSolmu);
390 }

```

4.2.2.18 puunPituus()

```

int puunPituus (
    PUU * pAlku )

```

Parameters

<i>pAlku</i>	Kohta, josta korkeus haetaan
--------------	------------------------------

Returns

int Palauttaa solmun korkeuden

Definition at line 293 of file binaaripuu.c.

```
293     {
294         // Tarkistetaan, onko Solmu Null
295         if (pAlku == NULL) {
296             return (0);
297         }
298
299         // Palauttaa solmun korkeuden
300         return (pAlku->iPituus);
301     }
```

4.2.2.19 suurempiLukuVertailu()

```
int suurempiLukuVertailu (
    int iLuku1,
    int iLuku2 )
```

Parameters

<i>iLuku1</i>	Ensimmäinen verrattava kokonaisluku.
<i>iLuku2</i>	Toinen verrattava kokonaisluku.

Returns

int Palauttaa suuremman luvun.

Definition at line 310 of file binaaripuu.c.

```
310     {
311         int iPalautus = 0;
312
313         // Verrataan kumpi luvuista on suurempi.
314         if (iLuku1 > iLuku2) {
315             iPalautus = iLuku1;
316         } else {
317             iPalautus = iLuku2;
318         }
319
320         // Palauttaa suuremman luvun.
321         return (iPalautus);
322     }
```

4.2.2.20 tasapainoitaPuu()

```
int tasapainoitaPuu (
    PUU * pAlku )
```

Parameters

<i>pAlku</i>	Solmu, josta tasapainokerroin halutaan
--------------	--

Returns

int

Definition at line 221 of file binaaripuu.c.

```

221      {
222      // Tarkistetaan, onko Null
223      if (pAlku == NULL) {
224          return (0);
225      }
226
227      // Palautetaan tasapainokerroin
228      return (puunPituus(pAlku->pVasen) - puunPituus(pAlku->pOikea));
229  }
```

4.2.2.21 vapautaMuistiPuu()

```

PUU* vapautaMuistiPuu (
    PUU * pJuuriSolmu )
```

Parameters

<i>pJuuriSolmu</i>	Puun juurisolmu
--------------------	-----------------

Returns

PUU*

Definition at line 39 of file binaaripuu.c.

```

39      {
40      // Muistin vapauttaminen
41      if (pJuuriSolmu != NULL) {
42          vapautaMuistiPuu(pJuuriSolmu->pVasen);
43          vapautaMuistiPuu(pJuuriSolmu->pOikea);
44          free(pJuuriSolmu);
45      }
46      pJuuriSolmu = NULL;
47      return (pJuuriSolmu);
48  }
```

4.2.2.22 vapautaMuistiRB()

```

RBSOLMU* vapautaMuistiRB (
    RBSOLMU * pJuuriSolmu )
```

Definition at line 28 of file punamustapuu.c.

```

28      {
29      if (pJuuriSolmu != NULL) {
30          vapautaMuistiRB(pJuuriSolmu->pVasen);
31          vapautaMuistiRB(pJuuriSolmu->pOikea);
32          free(pJuuriSolmu);
33      }
34      pJuuriSolmu = NULL;
35      return (pJuuriSolmu);
36  }
```

4.2.2.23 varaaMuistiaPuulle()

```
PUU* varaaMuistiaPuulle (
    char * pSolmu,
    int iArvo )
```

Parameters

<i>pSolmu</i>	Solmu, jolle muistia varataan
<i>iArvo</i>	Arvo, jolle muistia varataan

Returns

PUU*

Definition at line 16 of file binaaripuu.c.

```

16                                     {
17     PUU *pUusi = NULL;
18
19     // Muistin varaaminen
20     if ((pUusi = (PUU *)malloc(sizeof(PUU))) == NULL) {
21         perror("Muistin varaus epäonnistui, lopetetaan");
22         exit(0);
23     }
24
25     strcpy(pUusi->aNimi, pSolmu);
26     pUusi->iArvo = iArvo;
27     pUusi->iPituus = 1;
28     pUusi->pVasen = NULL;
29     pUusi->pOikea = NULL;
30     return (pUusi);
31 }
```

4.2.2.24 varaaMuistiaRB()

```

RBSOLMU* varaaMuistiaRB (
    char * pNimi,
    int iArvo )
```

Punamustapuu, ehkä oma header olisi kätevämpi, kun omat aliohjelmat oman structin takia.

Definition at line 11 of file punamustapuu.c.

```

11                                     {
12     RBSOLMU *pUusi = NULL;
13
14     if ((pUusi = (RBSOLMU *)malloc(sizeof(RBSOLMU))) == NULL) {
15         perror("Muistin varaus epäonnistui, lopetetaan");
16         exit(0);
17     }
18
19     pUusi->iVariBitti = 0; // uudet aina punaisia
20     strcpy(pUusi->aNimi, pNimi);
21     pUusi->iArvo = iArvo;
22     pUusi->pVasen = NULL;
23     pUusi->pOikea = NULL;
24     pUusi->pVanhempi = NULL;
25     return (pUusi);
26 }
```

4.2.2.25 vasenPuoli()

```

PUU* vasenPuoli (
    PUU * pAlku )
```

Puu järjestetään, jotta se on tasapainossa.

Parameters

<i>pAlku</i>	
--------------	--

Returns

PUU* Palauttaa solmun.

Definition at line 265 of file binaaripuu.c.

```

265         {
266     // Tarkistetaan, onko pAlku tai pAlku oikea puoli NULL
267     if ((pAlku->pOikea == NULL) || (pAlku == NULL)) {
268         return (pAlku);
269     }
270
271     // Muuttujien ja vakioiden alustaminen
272     PUU *pMuuttujal = pAlku->pOikea;
273     PUU *pMuuttuja2 = pMuuttujal->pVasen;
274
275     // Sijoitetaan arvoja
276     pMuuttujal->pVasen = pAlku;
277     pAlku->pOikea = pMuuttuja2;
278
279     // Verrataan ja sijoitetaan suuremmat korkeudet
280     pAlku->iPituus = suurempiLukuVertailu(puunPituus(pAlku->pVasen), puunPituus(pAlku->pOikea)) + 1;
281     pMuuttujal->iPituus =
282         suurempiLukuVertailu(puunPituus(pMuuttujal->pVasen), puunPituus(pMuuttujal->pOikea)) + 1;
283
284     return pMuuttujal;
285 }
```

4.3 haut.c File Reference

```

#include "haut.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- int [syvyyshaku](#) (char *pNimi, [PUU](#) *pAlku, int iArvo)
Tekee syvyysshaun numeroarvon pohjalta.
- void [tarkistaLoytyykoSyvyysshaulla](#) (char *aNimiKirjoitettava, [PUU](#) *pJuuriSolmu, int iArvo)
Tarkistaa, loytyyko etsitty arvo syvyysshaussa.
- void [leveyshaku](#) (char *pNimi, [PUU](#) *pJuuriSolmu, char *pHaku)
Tekee leveyshaun nimen pohjalta.
- [JONO](#) * [varaaMuistiaJonolle](#) ([PUU](#) *pSolmu)
Varaa muistia jonolle.
- [JONO](#) * [vapautaMuistiJono](#) ([JONO](#) *pAlku)
Vapauttaa jonon varaaman muistin.
- int [binaarihaku](#) (char *pNimi, [PUU](#) *pJuuriSolmu, int iArvo)
Binaarihaku perustuen numeroarvoon.

4.3.1 Function Documentation

4.3.1.1 binaarihaku()

```
int binaarihaku (
    char * pNimi,
    PUU * pJuuriSolmu,
    int iArvo )
```

Binaarihaku, joka tehdään käyttäjän antaman numeroarvon perusteella.

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan puun solmuun.
<i>iArvo</i>	Haettava numeroarvo.

Returns

int Palauttaa joko 0 tai 1, riippuen siitä, löytyykö numeroarvo binaaripuusta.

Definition at line 175 of file haut.c.

```
175                                     {
176
177     if (pJuuriSolmu == NULL) {
178         return (0);
179     }
180
181     kirjoitaTiedostoon(pNimi, pJuuriSolmu);
182
183     if (iArvo == pJuuriSolmu->iArvo) {
184         printf("Hakemasi arvo '%d' löytyi!\n", iArvo);
185         return (1);
186     } else if (iArvo < pJuuriSolmu->iArvo) {
187         return binaarihaku(pNimi, pJuuriSolmu->pVasen, iArvo);
188     } else {
189         return binaarihaku(pNimi, pJuuriSolmu->pOikea, iArvo);
190     }
191 }
```

4.3.1.2 leveyshaku()

```
void leveyshaku (
    char * pNimi,
    PUU * pJuuriSolmu,
    char * pHaku )
```

Tekee leveyshaun käyttäjän antaman nimen pohjalta. Kirjoittaa leveyshaun polun käyttäjän määrittelemään tiedostoon.

Parameters

<i>pNimi</i>	Käyttäjän antama kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>pHaku</i>	Osoitin haettavan nimen merkkijonoon.

Returns

void

Definition at line 73 of file haut.c.

```

73                                     {
74     // Leveyshaku nimen mukaan, kirjoitetaan käydyt solmut tiedostoon.
75     JONO *pAlku = varaaMuistiaJonolle(pJuuriSolmu);
76     JONO *pLoppu = pAlku;
77     JONO *pEdellinen = NULL;
78     int iVertailu = 0;
79     int iLoytyi = 0;
80
81     if (pJuuriSolmu == NULL) {
82         printf("Puu on tyhjä, luo puurakenne ennen leveyshakua.\n");
83         return;
84     }
85
86     while (pAlku != NULL) {
87         PUU *ptr = pAlku->pSolmu;
88         kirjoitaTiedostoon(pNimi, ptr);
89
90         iVertailu = strcmp(ptr->aNimi, pHaku);
91
92         if (iVertailu == 0) {
93             printf("Nimi '%s' löytyi puusta.\n", ptr->aNimi);
94             iLoytyi = 1;
95             break;
96         }
97
98         if (ptr->pVasen != NULL) {
99             pLoppu->pSeuraava = varaaMuistiaJonolle(ptr->pVasen);
100             pLoppu = pLoppu->pSeuraava;
101         }
102
103         if (ptr->pOikea != NULL) {
104             pLoppu->pSeuraava = varaaMuistiaJonolle(ptr->pOikea);
105             pLoppu = pLoppu->pSeuraava;
106         }
107
108         pEdellinen = pAlku;
109         pAlku = pAlku->pSeuraava;
110         free(pEdellinen);
111     }
112
113     if (iLoytyi == 0) {
114         printf("Hakemaasi nimeä ei löytynyt puusta.\n");
115     }
116     pAlku = vapautaMuistiJono(pAlku);
117     return;
118 }

```

4.3.1.3 syvyyshaku()

```

int syvyyshaku (
    char * pNimi,
    PUU * pAlku,
    int iArvo )

```

Tekee syvyyshaun käyttäjän antaman numeroarvon pohjalta. Kirjoittaa syvyyshaun polun käyttäjän määrittelemään tiedostoon.

Parameters

<i>pNimi</i>	Käyttäjän antama kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>iArvo</i>	Arvo, jota etsitään syvyyshaussa.

Returns

int Palauttaa arvon 0 tai 1, riippuen siitä, löytyyko etsitty arvo.

Definition at line 17 of file haut.c.

```

17                                     {
18     // Syvyyshaku arvon mukaan, kirjoitetaan käydyt solmut tiedostoon.
19     int iLoytyi = 0;
20
21     if (pAlku == NULL) {
22         return 0;
23     }
24
25     if (pAlku != NULL) {
26         kirjoitaTiedostoon(pNimi, pAlku);
27
28         if (pAlku->iArvo == iArvo) {
29             iLoytyi = 1;
30             printf("Arvo %d löytyi.\n", iArvo);
31         } else {
32             if (iLoytyi == 0) {
33                 iLoytyi = syvyyshaku(pNimi, pAlku->pVasen, iArvo);
34             }
35             if (iLoytyi == 0) {
36                 iLoytyi = syvyyshaku(pNimi, pAlku->pOikea, iArvo);
37             }
38         }
39     }
40     return (iLoytyi);
41 }
```

4.3.1.4 tarkistaLoytyykoSyvyyshauulla()

```

void tarkistaLoytyykoSyvyyshauulla (
    char * aNimiKirjoitettava,
    PUU * pJuuriSolmu,
    int iArvo )
```

Tarkistaa, löytyyko syvyysaussa etsitty arvo. Jos arvoa ei löydy, tulostaa tiedon siitä.

Parameters

<i>aNimiKirjoitettava</i>	Kayttajan antama kirjoitettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>iArvo</i>	Syvyysaussa haettava numeroarvo.

Returns

void

Definition at line 54 of file haut.c.

```

54                                     {
55     int iLoytyi = 0;
56     iLoytyi = syvyyshaku(aNimiKirjoitettava, pJuuriSolmu, iArvo);
57     if (iLoytyi == 0) {
58         printf("Arvoa %d ei löytynyt puusta.\n", iArvo);
59     }
60 }
```

4.3.1.5 vapautaMuistiJono()

```
JONO* vapautaMuistiJono (
    JONO * pAlku )
```

Vapauttaa jonon alkioita varten varatun muistin.

Parameters

<i>pAlku</i>	Osoitin jonon ensimmäiseen alkioon.
--------------	-------------------------------------

Returns

pAlku Palauttaa NULL, koska jono on tyhjä.

Definition at line 151 of file haut.c.

```
151                                     {
152     // Jonon muistin vapauttaminen.
153     JONO *ptr = pAlku;
154     JONO *pSeuraava = NULL;
155
156     while (ptr != NULL) {
157         pSeuraava = ptr->pSeuraava;
158         free(ptr);
159         ptr = pSeuraava;
160     }
161     pAlku = NULL;
162     return (pAlku);
163 }
```

4.3.1.6 varaaMuistiaJonolle()

```
JONO* varaaMuistiaJonolle (
    PUU * pSolmu )
```

Varaa muistia jonon uudelle alkiolle leveyshakua varten.

Parameters

<i>pSolmu</i>	Osoitin kasiteltavaan puun solmuun.
---------------	-------------------------------------

Returns

pUusi Osoitin uuteen jonon alkioon.

Definition at line 128 of file haut.c.

```
128                                     {
129     JONO *pUusi = NULL;
130
131     // Muistin varaaminen jonolle.
132     if ((pUusi = (JONO *)malloc(sizeof(JONO))) == NULL) {
133         perror("Muistin varaus epäonnistui, lopetetaan");
134         exit(0);
135     }
136
137     pUusi->pSolmu = pSolmu;
138     pUusi->pSeuraava = NULL;
139
140     return (pUusi);
141 }
```

4.4 haut.h File Reference

```
#include "binaaripuu.h"
```

Classes

- struct [jono](#)

Typedefs

- typedef struct [jono](#) [JONO](#)

Functions

- int [syvyyshaku](#) (char *pKirjoitaTiedostoNimi, [PUU](#) *pJuuriSolmu, int iArvo)
Tekee syvyysshaun numeroarvon pohjalta.
- void [tarkistaLoytvykoSyvyysshaulla](#) (char *aNimiKirjoitettava, [PUU](#) *pJuuriSolmu, int iArvo)
Tarkistaa, loytyyko etsitty arvo syvyysshaussa.
- void [leveyshaku](#) (char *pKirjoitaTiedostoNimi, [PUU](#) *pAlku, char *pHaku)
Tekee leveysshaun nimen pohjalta.
- [JONO](#) * [varaaMuistiaJonolle](#) ([PUU](#) *pSolmu)
Varaa muistia jonolle.
- [JONO](#) * [vapautaMuistiJono](#) ([JONO](#) *pAlku)
Vapauttaa jonon varaaman muistin.
- int [binaarihaku](#) (char *pNimi, [PUU](#) *pJuuriSolmu, int iArvo)
Binaarihaku perustuen numeroarvoon.

4.4.1 Typedef Documentation

4.4.1.1 JONO

```
typedef struct jono JONO
```

4.4.2 Function Documentation

4.4.2.1 binaarihaku()

```
int binaarihaku (  
    char * pNimi,  
    PUU * pJuuriSolmu,  
    int iArvo )
```

Binaarihaku, joka tehdään käyttäen antaman numeroarvon perusteella.

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Osoitin kasiteltavaan puun solmuun.
<i>iArvo</i>	Haettava numeroarvo.

Returns

int Palauttaa joko 0 tai 1, riippuen siitä, löytyykö numeroarvo binaaripuusta.

Definition at line 175 of file haut.c.

```

175                                     {
176
177     if (pJuuriSolmu == NULL) {
178         return (0);
179     }
180
181     kirjoitaTiedostoon(pNimi, pJuuriSolmu);
182
183     if (iArvo == pJuuriSolmu->iArvo) {
184         printf("Hakemasi arvo '%d' löytyi!\n", iArvo);
185         return (1);
186     } else if (iArvo < pJuuriSolmu->iArvo) {
187         return binaarihaku(pNimi, pJuuriSolmu->pVasen, iArvo);
188     } else {
189         return binaarihaku(pNimi, pJuuriSolmu->pOikea, iArvo);
190     }
191 }
```

4.4.2.2 leveyshaku()

```

void leveyshaku (
    char * pNimi,
    PUU * pJuuriSolmu,
    char * pHaku )
```

Tekee leveyshaun käyttäjän antaman nimen pohjalta. Kirjoittaa leveyshaun polun käyttäjän määrittelemään tiedostoon.

Parameters

<i>pNimi</i>	Käyttäjän antama kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>pHaku</i>	Osoitin haettavan nimen merkkijonoon.

Returns

void

Definition at line 73 of file haut.c.

```

73                                     {
74     // Leveyshaku nimen mukaan, kirjoitetaan käydyt solmut tiedostoon.
75     JONO *pAlku = varaaMuistiaJonolle(pJuuriSolmu);
76     JONO *pLoppu = pAlku;
77     JONO *pEdellinen = NULL;
78     int iVertailu = 0;
79     int iLöytyi = 0;
80 }
```

```

81     if (pJuuriSolmu == NULL) {
82         printf("Puu on tyhjä, luo puurakenne ennen leveyshakua.\n");
83         return;
84     }
85
86     while (pAlku != NULL) {
87         PUU *ptr = pAlku->pSolmu;
88         kirjoitaTiedostoon(pNimi, ptr);
89
90         iVertailu = strcmp(ptr->aNimi, pHaku);
91
92         if (iVertailu == 0) {
93             printf("Nimi '%s' löytyi puusta.\n", ptr->aNimi);
94             iLoytyi = 1;
95             break;
96         }
97
98         if (ptr->pVasen != NULL) {
99             pLoppu->pSeuraava = varaaMuistiaJonolle(ptr->pVasen);
100             pLoppu = pLoppu->pSeuraava;
101         }
102
103         if (ptr->pOikea != NULL) {
104             pLoppu->pSeuraava = varaaMuistiaJonolle(ptr->pOikea);
105             pLoppu = pLoppu->pSeuraava;
106         }
107
108         pEdellinen = pAlku;
109         pAlku = pAlku->pSeuraava;
110         free(pEdellinen);
111     }
112
113     if (iLoytyi == 0) {
114         printf("Hakemaasi nimeä ei löytynyt puusta.\n");
115     }
116     pAlku = vapautaMuistiJono(pAlku);
117     return;
118 }

```

4.4.2.3 syvyyshaku()

```

int syvyyshaku (
    char * pNimi,
    PUU * pAlku,
    int iArvo )

```

Tekee syvyysshaun käyttäjän antaman numeroarvon pohjalta. Kirjoittaa syvyysshaun polun käyttäjän määrittelemään tiedostoon.

Parameters

<i>pNimi</i>	Käyttäjän antama kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>iArvo</i>	Arvo, jota etsitään syvyysshaussa.

Returns

int Palauttaa arvon 0 tai 1, riippuen siitä, löytyykö etsitty arvo.

Definition at line 17 of file haut.c.

```

17     {
18         // Syvyyshaku arvon mukaan, kirjoitetaan käydyt solmut tiedostoon.
19         int iLoytyi = 0;
20
21         if (pAlku == NULL) {
22             return 0;
23         }

```

```

24
25     if (pAlku != NULL) {
26         kirjoitaTiedostoon(pNimi, pAlku);
27
28         if (pAlku->iArvo == iArvo) {
29             iLoytyi = 1;
30             printf("Arvo %d löytyi.\n", iArvo);
31         } else {
32             if (iLoytyi == 0) {
33                 iLoytyi = syvyyshaku(pNimi, pAlku->pVasen, iArvo);
34             }
35             if (iLoytyi == 0) {
36                 iLoytyi = syvyyshaku(pNimi, pAlku->pOikea, iArvo);
37             }
38         }
39     }
40     return (iLoytyi);
41 }

```

4.4.2.4 tarkistaLoytzykoSyvyyshauulla()

```

void tarkistaLoytzykoSyvyyshauulla (
    char * aNimiKirjoitettava,
    PUU * pJuuriSolmu,
    int iArvo )

```

Tarkistaa, löytyykö syvyysaussa etsitty arvo. Jos arvoa ei löydy, tulostaa tiedon siitä.

Parameters

<i>aNimiKirjoitettava</i>	Kayttajan antama kirjoitettavan tiedoston nimi.
<i>pJuuriSolmu</i>	Osoitin puu tietueen alkuun, eli ensimmäiseen solmuun.
<i>iArvo</i>	Syvyysaussa haettava numeroarvo.

Returns

void

Definition at line 54 of file haut.c.

```

54
55     int iLoytyi = 0;
56     iLoytyi = syvyyshaku(aNimiKirjoitettava, pJuuriSolmu, iArvo);
57     if (iLoytyi == 0) {
58         printf("Arvoa %d ei löytynyt puusta.\n", iArvo);
59     }
60 }

```

4.4.2.5 vapautaMuistiJono()

```

JONO* vapautaMuistiJono (
    JONO * pAlku )

```

Vapauttaa jonon alkioita varten varatun muistin.

Parameters

<i>pAlku</i>	Osoitin jonon ensimmäiseen alkioon.
--------------	-------------------------------------

Returns

pAlku Palauttaa NULL, koska jono on tyhjä.

Definition at line 151 of file haut.c.

```
151                                     {
152     // Jonon muistin vapauttaminen.
153     JONO *ptr = pAlku;
154     JONO *pSeuraava = NULL;
155
156     while (ptr != NULL) {
157         pSeuraava = ptr->pSeuraava;
158         free(ptr);
159         ptr = pSeuraava;
160     }
161     pAlku = NULL;
162     return (pAlku);
163 }
```

4.4.2.6 varaaMuistiaJonolle()

```
JONO* varaaMuistiaJonolle (
    PUU * pSolmu )
```

Varaa muistia jonon uudelle alkionle leveyshakua varten.

Parameters

<i>pSolmu</i>	Osoitin kasiteltavaan puun solmuun.
---------------	-------------------------------------

Returns

pUusi Osoitin uuteen jonon alkioon.

Definition at line 128 of file haut.c.

```
128                                     {
129     JONO *pUusi = NULL;
130
131     // Muistin varaaminen jonolle.
132     if ((pUusi = (JONO *)malloc(sizeof(JONO))) == NULL) {
133         perror("Muistin varaus epäonnistui, lopetetaan");
134         exit(0);
135     }
136
137     pUusi->pSolmu = pSolmu;
138     pUusi->pSeuraava = NULL;
139
140     return (pUusi);
141 }
```

4.5 linkitettylista.c File Reference

```
#include "linkitettylista.h"
#include <stdio.h>
```



```
#include <stdlib.h>
#include <string.h>
```

Functions

- **TIEDOT * lue** (char *pNimi, **TIEDOT *pAlku**)
Lukee tiedoston.
- **TIEDOT * varaaMuistia** (**TIEDOT *pAlku**, char *pNimi, int iMaara)
Varaa muistin ja lisää uuden solmun kaksisuuntaisen linkitetyn listan.
- **TIEDOT * vapautaMuisti** (**TIEDOT *pAlku**)
Vapauttaa linkitetyn listan varaaman muistin.
- void **kirjoitaTiedostoAlustaLoppuun** (char *pNimi, **TIEDOT *pAlku**)
Kirjoittaa linkitetyn listan tiedostoon alusta loppuun.
- void **kirjoitaTiedostoLopustaAlkuun** (char *pNimi, **TIEDOT *pAlku**)
Kirjoittaa linkitetyn listan tiedostoon lopusta alkuun.
- **TIEDOT * lisaaListaan** (**TIEDOT *pAlku**, int iIndeksi, char *pNimi, int iArvo)
Lisää linkitettyyn listaan käyttäjän syöttämät tiedot.
- **TIEDOT * poistaListastaLkmPerusteella** (**TIEDOT *pAlku**, int iLKM)
Poistetaan linkitetystä listasta alkio lukumäärän perusteella.
- **TIEDOT * poistaListastaNimenPerusteella** (**TIEDOT *pAlku**, char *pNimi)
Poistetaan linkitetystä listasta alkio nimen perusteella.
- int **useammallaAlkiollaSamaLKM** (**TIEDOT *pAlku**, int iLKM)
Etsii kaikki alkiot, joiden nimien lukumäärä on käyttäjän antama luku.
- int **laskeListanPituus** (**TIEDOT *pAlku**)
Laskee linkitetyn listan pituuden.
- **TIEDOT * halkaise** (**TIEDOT *pAlku**)
Halkaisee linkitetyn listan kahteen puoliskoon.
- **TIEDOT * lomitus** (**TIEDOT *p1**, **TIEDOT *p2**)
Lomittaa (merge) kaksi lajiteltua listaa yhdeksi.
- **TIEDOT * lomitusLajittelu** (**TIEDOT *pAlku**)
Lajittelee listan rekursiivisella lomituslajittelulla.

4.5.1 Function Documentation

4.5.1.1 halkaise()

```
TIEDOT* halkaise (
    TIEDOT * pAlku )
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
--------------	-----------------------------------

Returns

TIEDOT* Osoitin listan jälkimmäisen puoliskon alkuun.

Definition at line 377 of file linkitettylista.c.

```

377     {
378         TIEDOT *p1 = NULL, *p2 = NULL;
379         int i;
380         int iPituus = 0;
381         int iKeskipiste = 0;
382
383         iPituus = laskeListanPituus(pAlku);
384         iKeskipiste = iPituus / 2 - 1;
385
386         p1 = pAlku;
387         for (i = 0; i < iKeskipiste; i++) {
388             p1 = p1->pSeuraava;
389         }
390
391         // Halkaistaan lista kahtia, p1 päättyy siihen mistä p2 alkaa
392         p2 = p1->pSeuraava;
393         p1->pSeuraava = NULL;
394         if (p2 != NULL) {
395             p2->pEdellinen = NULL;
396         }
397
398         // palautetaan osoitin toisen listan alkuun
399         return p2;
400     }

```

4.5.1.2 kirjoitaTiedostoAlustaLoppuun()

```

void kirjoitaTiedostoAlustaLoppuun (
    char * pNimi,
    TIEDOT * pAlku )

```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

void

Definition at line 129 of file linkitettylista.c.

```

129     {
130         FILE *Tiedosto = NULL;
131         TIEDOT *ptr = NULL;
132
133         if (pAlku == NULL) {
134             printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
135             return;
136         }
137
138         // Kirjoitus tiedoston avaaminen
139         if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
140             perror("Tiedoston avaaminen epäonnistui, lopetetaan");
141             exit(0);
142         }
143
144         // Tiedostoon kirjoittaminen
145         ptr = pAlku;
146         while (ptr != NULL) {
147             fprintf(Tiedosto, "%s,%d\n", ptr->aSukunimi, ptr->iYhteensa);
148             ptr = ptr->pSeuraava;
149         }

```

```

150
151     // Tiedoston sulkeminen
152     fclose(Tiedosto);
153     printf("Tiedosto %s kirjoitettu.\n", pNimi);
154     return;
155 }

```

4.5.1.3 kirjoitaTiedostoLopustaAlkuun()

```

void kirjoitaTiedostoLopustaAlkuun (
    char * pNimi,
    TIEDOT * pAlku )

```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

void

Definition at line 164 of file linkitettylista.c.

```

164
165     FILE *Tiedosto = NULL;
166     TIEDOT *ptr = pAlku;
167
168     if (pAlku == NULL) {
169         printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
170         return;
171     }
172
173     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
174         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
175         exit(0);
176     }
177
178     // Mennään listan loppuun.
179     while (ptr->pSeuraava != NULL) {
180         ptr = ptr->pSeuraava;
181     }
182
183     // Lopusta alkuun, kirjoitetaan tiedostoon.
184     while (ptr != NULL) {
185         fprintf(Tiedosto, "%s,%d\n", ptr->aSukunimi, ptr->iYhteensa);
186         ptr = ptr->pEdellinen;
187     }
188
189     fclose(Tiedosto);
190     printf("Tiedosto %s kirjoitettu.\n", pNimi);
191     return;
192 }

```

4.5.1.4 laskeListanPituus()

```

int laskeListanPituus (
    TIEDOT * pAlku )

```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
--------------	-----------------------------------

Returns

int Linkitetyn listan alkioden lukumäärä.

Definition at line 362 of file linkitettylista.c.

```

362                                     {
363     int i = 0;
364     while (pAlku != NULL) {
365         i++;
366         pAlku = pAlku->pSeuraava;
367     }
368     return i;
369 }
```

4.5.1.5 lisääListaan()

```

TIEDOT* lisääListaan (
    TIEDOT * pAlku,
    int iIndeksi,
    char * pNimi,
    int iArvo )
```

Lisaa käyttäjän haluamaan kohtaan (indeksi) tiedot lisattavasta nimestä ja niiden lukumaarasta.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iIndeksi</i>	Käyttäjän antama indeksi, eli kohta, johon tietue lisataan linkitettyssä listassa. Jos lista on tyhjä, lisataan automaattisesti ensimmäiseksi elementiksi. Jos indeksi on suurempi kuin listan elementtien määrä, lisataan automaattisesti viimeiseksi.
<i>pNimi</i>	Lisattava nimi.
<i>iArvo</i>	Lisattavan nimen lukumaara.

Returns

pAlku Osoitin listan nykyiseen alkuun.

Definition at line 207 of file linkitettylista.c.

```

207                                     {
208     TIEDOT *pUusi = NULL;
209     TIEDOT *ptr = pAlku;
210     int i = 0;
211
212     // Varataan muistia uudelle linkitetyn listan solmulle.
213     if ((pUusi = (TIEDOT *)malloc(sizeof(TIEDOT))) == NULL) {
214         perror("Muistin varaus epäonnistui, lopetetaan");
215         exit(0);
216     }
217
218     // Lisataan tiedot tietueeseen.
219     strcpy(pUusi->aSukunimi, pNimi);
220     pUusi->iYhteensa = iArvo;
221 }
```

```

222 // Jos lista on tyhja, lisataan uusi solmu ensimmäiseksi.
223 if (pAlku == NULL) {
224     pUusi->pEdellinen = NULL;
225     pUusi->pSeuraava = NULL;
226     pAlku = pUusi;
227     return (pAlku);
228 } else {
229     // Jos lisataan listan alkuun, eli indeksi = 0.
230     if (iIndeksi == 0) {
231         pUusi->pEdellinen = NULL;
232         pUusi->pSeuraava = pAlku;
233         pAlku->pEdellinen = pUusi;
234         pAlku = pUusi;
235         return (pAlku);
236     }
237
238     // Etsitaan listalta oikea kohta, johon tiedot lisataan.
239     while (ptr->pSeuraava != NULL && i < (iIndeksi - 1)) {
240         ptr = ptr->pSeuraava;
241         i++;
242     }
243     // Lisataan tiedot oikeaan kohtaan linkitettyä listaa.
244     // Paivitetään osoittimet osoittamaan oikeisiin kohtiin.
245     pUusi->pSeuraava = ptr->pSeuraava;
246     pUusi->pEdellinen = ptr;
247
248     if (ptr->pSeuraava != NULL) {
249         ptr->pSeuraava->pEdellinen = pUusi;
250     }
251
252     ptr->pSeuraava = pUusi;
253 }
254 return (pAlku);
255 }

```

4.5.1.6 lomitus()

```

TIEDOT* lomitus (
    TIEDOT * p1,
    TIEDOT * p2 )

```

Parameters

<i>p1</i>	Osoitin ensimmäisen lajitellun listan alkuun.
<i>p2</i>	Osoitin jälkimmäisen lajitellun listan alkuun.

Returns

TIEDOT* yhdistetty lajiteltu lista.

Definition at line 409 of file linkitettylista.c.

```

409 {
410     // Jos kumpikaan lista on tyhjä, palautetaan toinen.
411     if (p1 == NULL)
412         return p2;
413     if (p2 == NULL)
414         return p1;
415
416     TIEDOT *p3 = NULL; // Uuden listan alku.
417     TIEDOT *p3Loppu = NULL; // Uuden lista loppu, jotta loppuun lisääminen onnistuu helposti.
418
419     // Käydään molempia listoja läpi:
420     while (p1 != NULL && p2 != NULL) {
421         if (p1->iYhteensa > p2->iYhteensa) {
422             TIEDOT *ptr = p2; // p2[0] talteen
423             // Poistetaan p2[0] listasta.
424             p2 = p2->pSeuraava;
425             if (p2 != NULL) {
426                 p2->pEdellinen = NULL;

```

```

427         }
428         // Lisätään p2[0] uuden listan loppuun.
429         ptr->pSeuraava = NULL;
430         ptr->pEdellinen = p3Loppu;
431         if (p3Loppu != NULL) {
432             p3Loppu->pSeuraava = ptr;
433         } else {
434             p3 = ptr; // ensimmäinen alkio, jos lista tyhjä
435         }
436         p3Loppu = ptr; // päivitetään loppu.
437     } else {
438         TIEDOT *ptr = p1; // p1[0] talteen
439         p1 = p1->pSeuraava;
440         // Poistetaan p1[0] listasta.
441         if (p1 != NULL) {
442             p1->pEdellinen = NULL;
443         }
444         // Lisätään p1[0] uuden listan loppuun.
445         ptr->pSeuraava = NULL;
446         ptr->pEdellinen = p3Loppu;
447         if (p3Loppu != NULL) {
448             p3Loppu->pSeuraava = ptr;
449         } else {
450             p3 = ptr; // ensimmäinen alkio, jos lista tyhjä
451         }
452         p3Loppu = ptr; // päivitetään loppu.
453     }
454 }
455
456 // Tässä kohtaa joko p1 tai p2 on tyhjä.
457
458 // Lisätään loput p1:stä uuden listan loppuun.
459 while (p1 != NULL) {
460     TIEDOT *ptr = p1;
461     p1 = p1->pSeuraava;
462     if (p1 != NULL) {
463         p1->pEdellinen = NULL;
464     }
465     ptr->pSeuraava = NULL;
466     ptr->pEdellinen = p3Loppu;
467     if (p3Loppu != NULL) {
468         p3Loppu->pSeuraava = ptr;
469     } else {
470         p3 = ptr;
471     }
472     p3Loppu = ptr;
473 }
474
475 // Lisätään loput p2:stä uuden listan loppuun.
476 while (p2 != NULL) {
477     TIEDOT *ptr = p2;
478     p2 = p2->pSeuraava;
479     if (p2 != NULL) {
480         p2->pEdellinen = NULL;
481     }
482     ptr->pSeuraava = NULL;
483     ptr->pEdellinen = p3Loppu;
484     if (p3Loppu != NULL) {
485         p3Loppu->pSeuraava = ptr;
486     } else {
487         p3 = ptr;
488     }
489     p3Loppu = ptr;
490 }
491 return p3;
492 }

```

4.5.1.7 lomitusLajittelu()

```

TIEDOT* lomitusLajittelu (
    TIEDOT * pAlku )

```

Parameters

<i>pAlku</i>	Osoitin lajiteltavan linkitetyn listan alkuun.
--------------	--

Returns

TIEDOT* Lajiteltu lista.

Definition at line 500 of file linkitettylista.c.

```

500                                     {
501     TIEDOT *p1 = NULL, *p2 = NULL;
502
503     // Jos lista on tyhjä tai sisältää vain yhden alkion
504     if (pAlku == NULL || pAlku->pSeuraava == NULL) {
505         return pAlku;
506     }
507
508     // Jaetaan lista kahtia
509     p2 = halkaise(pAlku);
510     p1 = pAlku;
511
512     // Lajitellaan molemmat puolikkaat
513     p1 = lomituspLajittelu(p1);
514     p2 = lomituspLajittelu(p2);
515
516     // Yhdistetään lajitellut puolikkaat
517     return lomitusp(p1, p2);
518 }
```

4.5.1.8 lue()

```

TIEDOT* lue (
    char * pNimi,
    TIEDOT * pAlku )
```

Avaa ja lukee käyttäjän syötteen mukaisen tiedoston. Kutsuu muistinvaraus aliohjelman muistin varaamiseksi.

Parameters

<i>pNimi</i>	Luettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

pAlku Uusi osoitin linkitetyn listan alkuun.

Definition at line 18 of file linkitettylista.c.

```

18                                     {
19     FILE *Tiedosto = NULL;
20     char aRivi[LEN] = "";
21     char *p1 = NULL, *p2 = NULL;
22     int iOtsikko = 0;
23     int iMaara = 0;
24
25     /* Tiedoston avaaminen. */
26
27     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
28         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
29         exit(0);
30     }
31
32     /* Tiedoston lukeminen */
33
34     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
35
36         aRivi[strlen(aRivi) - 1] = '\0';
37
38         if (iOtsikko == 0) {
39             iOtsikko = 1;
40             continue;
41         }
42     }
```

```

41     }
42
43     if ((p1 = strtok(aRivi, ";")) == NULL) {
44         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
45         exit(0);
46     }
47
48     if ((p2 = strtok(NULL, ";")) == NULL) {
49         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
50         exit(0);
51     }
52
53     iMaara = atoi(p2);
54
55     /* Muistin varaaminen ja linkitetyn listan luominen. */
56
57     pAlku = varaaMuistia(pAlku, p1, iMaara);
58 }
59 /* Tiedoston sulkeminen. */
60 fclose(Tiedosto);
61 printf("Tiedosto '%s' luettu.\n", pNimi);
62 return (pAlku);
63 }

```

4.5.1.9 poistaListastaLkmPeruusteella()

```

TIEDOT* poistaListastaLkmPeruusteella (
    TIEDOT * pAlku,
    int iLKM )

```

Käyttäjä on antanut lukumäärän. Alkio, jossa on tämä lukumäärä poistetaan.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iLKM</i>	Käyttäjän antama lukumäärä. Alkio, jossa on tämä lukumäärä poistetaan.

Returns

TIEDOT* Palautetaan linkitettylista, josta on poistettu alkio.

Definition at line 264 of file linkitettylista.c.

```

264
265     TIEDOT *ptr = pAlku;
266     TIEDOT *pEdellinen = NULL;
267
268     // Tarkistetaan, onko poistettava alkio ensimmäinen.
269     if ((ptr != NULL) && (ptr->iYhteensa == iLKM)) {
270         // Sijoitetaan osoitin osoittamaan seuraavaan alkioon.
271         pAlku = ptr->pSeuraava;
272         free(ptr);
273         printf("Alkio poistettu.\n");
274
275         // Jos poistettava alkio ei ole ensimmäinen.
276     } else {
277         // Etsitään kohtaa, jossa poistettava alkio on.
278         while (ptr != NULL) {
279             if (ptr->iYhteensa == iLKM) {
280                 pEdellinen->pSeuraava = ptr->pSeuraava;
281                 free(ptr);
282                 printf("Alkio poistettu.\n");
283                 break;
284             }
285             pEdellinen = ptr;
286             ptr = ptr->pSeuraava;
287         }
288     }
289

```



```

290     return (pAlku);
291 }

```

4.5.1.10 poistaListastaNimenPerusteella()

```

TIEDOT* poistaListastaNimenPerusteella (
    TIEDOT * pAlku,
    char * pNimi )

```

Käyttäjä on antanut nimen. Alkio, jossa on tämä nimi poistetaan.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>pNimi</i>	Käyttäjän antama nimi. Alkio, jossa on tämä nimi poistetaan.

Returns

TIEDOT* TIEDOT* Palautetaan linkitettylista, josta on poistettu alkio.

Definition at line 300 of file linkitettylista.c.

```

300                                                     {
301     TIEDOT *ptr = pAlku;
302     TIEDOT *pEdellinen = NULL;
303
304     // Tarkistetaan, onko poistettava alkio ensimmäinen.
305     if ((ptr != NULL) && (strcmp(ptr->aSukunimi, pNimi) == 0)) {
306         // Sijoitetaan osoitin osoittamaan seuraavaan alkioon.
307         pAlku = ptr->pSeuraava;
308         free(ptr);
309         printf("Alkio poistettu.\n");
310
311         // Jos poistettava alkio ei ole ensimmäinen.
312     } else {
313         // Etsitään kohtaa, jossa poistettava alkio on.
314         while (ptr != NULL) {
315             if (strcmp(ptr->aSukunimi, pNimi) == 0) {
316                 pEdellinen->pSeuraava = ptr->pSeuraava;
317                 free(ptr);
318                 printf("Alkio poistettu.\n");
319                 break;
320             }
321             pEdellinen = ptr;
322             ptr = ptr->pSeuraava;
323         }
324     }
325     return (pAlku);
326 }

```

4.5.1.11 useammallaAlkiollaSamaLKM()

```

int useammallaAlkiollaSamaLKM (
    TIEDOT * pAlku,
    int iLKM )

```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iLKM</i>	Käyttäjän antama lukumäärä, joiden määrä selvitetään.

Returns

int Palauttaa luvun, jossa on kaikki alkiot, joissa on käyttäjän antama luku.

Definition at line 335 of file linkitettylista.c.

```

335                                     {
336     int iSamoja = 0;
337     TIEDOT *ptr = NULL;
338     ptr = pAlku;
339
340     // Käydään linkitettylista läpi
341     while (ptr != NULL) {
342         // Lisätään summaan yksi, jos linkitetyn listan alkion lukumäärä on sama kuin käyttäjän
343         // antama luku.
344         if (ptr->iYhteensa == iLKM) {
345             iSamoja++;
346         }
347         ptr = ptr->pSeuraava;
348     }
349
350     return (iSamoja);
351 }
```

4.5.1.12 vapautaMuisti()

```

TIEDOT* vapautaMuisti (
    TIEDOT * pAlku )
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
--------------	-----------------------------------

Returns

pAlku Osoitin, joka on NULL, koska lista on tyhjä.

Definition at line 110 of file linkitettylista.c.

```

110                                     {
111     /* Muistin vapauttaminen. */
112     TIEDOT *ptr = pAlku;
113     while (ptr != NULL) {
114         pAlku = ptr->pSeuraava;
115         free(ptr);
116         ptr = pAlku;
117     }
118     pAlku = NULL;
119     return (pAlku);
120 }
```

4.5.1.13 varaaMuistia()

```

TIEDOT* varaaMuistia (
    TIEDOT * pAlku,
    char * pNimi,
    int iMaara )
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>pNimi</i>	Merkkijono solmun nimeksi.
<i>iMaara</i>	Luku, joka asetetaan solmuun maaraksi.

Returns

pAlku Uusi osoitin linkitetyn listan alkuun.

Definition at line 73 of file linkitettylista.c.

```

73                                     {
74     TIEDOT *pUusi = NULL;
75     TIEDOT *ptr = NULL;
76
77     /* Muistin varaaminen. */
78
79     if ((pUusi = (TIEDOT *)malloc(sizeof(TIEDOT))) == NULL) {
80         perror("Muistin varaus epäonnistui, lopetetaan");
81         exit(0);
82     }
83
84     /* Kaksisuuntaisen linkitetyn listan luominen. */
85
86     strcpy(pUusi->aSukunimi, pNimi);
87     pUusi->iYhteensa = iMaara;
88     pUusi->pSeuraava = NULL;
89
90     if (pAlku == NULL) {
91         pUusi->pEdellinen = NULL;
92         pAlku = pUusi;
93     } else {
94         ptr = pAlku;
95         while (ptr->pSeuraava != NULL) {
96             ptr = ptr->pSeuraava;
97         }
98         ptr->pSeuraava = pUusi;
99         pUusi->pEdellinen = ptr;
100     }
101     return (pAlku);
102 }
```

4.6 linkitettylista.h File Reference

```
#include "yleiset.h"
```

Classes

- struct [tiedot](#)

Typedefs

- typedef struct [tiedot](#) TIEDOT

Functions

- `TIEDOT * lue` (char *pNimi, `TIEDOT *pAlku`)
Lukee tiedoston.
- `TIEDOT * varaaMuistia` (`TIEDOT *pAlku`, char *pSukunimi, int iMaara)
Varaa muistin ja lisää uuden solmun kaksisuuntaisen linkitetyn listan.
- `TIEDOT * vapautaMuisti` (`TIEDOT *pAlku`)
Vapauttaa linkitetyn listan varaaman muistin.
- void `kirjoitaTiedostoAlustaLoppuun` (char *pKirjoitaTiedostoNimi, `TIEDOT *pAlku`)
Kirjoittaa linkitetyn listan tiedostoon alusta loppuun.
- void `kirjoitaTiedostoLopustaAlkuun` (char *pKirjoitaTiedostoNimi, `TIEDOT *pAlku`)
Kirjoittaa linkitetyn listan tiedostoon lopusta alkuun.
- `TIEDOT * lisaaListaan` (`TIEDOT *pAlku`, int iIndeksi, char *pNimi, int iArvo)
Lisää linkitettyyn listaan käyttäjän syöttämät tiedot.
- `TIEDOT * poistaListastaLkmPerusteella` (`TIEDOT *pAlku`, int iLKM)
Poistetaan linkitetystä listasta alkio lukumäärän perusteella.
- int `useammallaAlkiollaSamaLKM` (`TIEDOT *pAlku`, int iLKM)
Etsii kaikki alkiot, joiden nimien lukumäärä on käyttäjän antama luku.
- `TIEDOT * poistaListastaNimenPerusteella` (`TIEDOT *pAlku`, char *pNimi)
Poistetaan linkitetystä listasta alkio nimen perusteella.
- int `laskeListanPituus` (`TIEDOT *pAlku`)
Laskee linkitetyn listan pituuden.
- `TIEDOT * halkaise` (`TIEDOT *pAlku`)
Halkaisee linkitetyn listan kahteen puoliskoon.
- `TIEDOT * lomit` (`TIEDOT *p1`, `TIEDOT *p2`)
Lomittaa (merge) kaksi lajiteltua listaa yhdeksi.
- `TIEDOT * lomitLajittelu` (`TIEDOT *pAlku`)
Lajittelee listan rekursiivisella lomitushajittelulla.

4.6.1 Typedef Documentation

4.6.1.1 TIEDOT

```
typedef struct tiedot TIEDOT
```

4.6.2 Function Documentation

4.6.2.1 halkaise()

```
TIEDOT* halkaise (
    TIEDOT * pAlku )
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
--------------	-----------------------------------

Returns

TIEDOT* Osoitin listan jälkimmäisen puoliskon alkuun.

Definition at line 377 of file linkitettylista.c.

```

377     {
378         TIEDOT *p1 = NULL, *p2 = NULL;
379         int i;
380         int iPituus = 0;
381         int iKeskipiste = 0;
382
383         iPituus = laskeListanPituus(pAlku);
384         iKeskipiste = iPituus / 2 - 1;
385
386         p1 = pAlku;
387         for (i = 0; i < iKeskipiste; i++) {
388             p1 = p1->pSeuraava;
389         }
390
391         // Halkaistaan lista kahtia, p1 päättyy siihen mistä p2 alkaa
392         p2 = p1->pSeuraava;
393         p1->pSeuraava = NULL;
394         if (p2 != NULL) {
395             p2->pEdellinen = NULL;
396         }
397
398         // palautetaan osoitin toisen listan alkuun
399         return p2;
400     }

```

4.6.2.2 kirjoitaTiedostoAlustaLoppuun()

```

void kirjoitaTiedostoAlustaLoppuun (
    char * pNimi,
    TIEDOT * pAlku )

```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

void

Definition at line 129 of file linkitettylista.c.

```

129     {
130         FILE *Tiedosto = NULL;
131         TIEDOT *ptr = NULL;
132
133         if (pAlku == NULL) {
134             printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
135             return;
136         }
137
138         // Kirjoitus tiedoston avaaminen
139         if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
140             perror("Tiedoston avaaminen epäonnistui, lopetetaan");

```

```

141         exit(0);
142     }
143
144     // Tiedostoon kirjoittaminen
145     ptr = pAlku;
146     while (ptr != NULL) {
147         fprintf(Tiedosto, "%s,%d\n", ptr->aSukunimi, ptr->iYhteensa);
148         ptr = ptr->pSeuraava;
149     }
150
151     // Tiedoston sulkeminen
152     fclose(Tiedosto);
153     printf("Tiedosto %s kirjoitettu.\n", pNimi);
154     return;
155 }

```

4.6.2.3 kirjoitaTiedostoLopustaAlkuun()

```

void kirjoitaTiedostoLopustaAlkuun (
    char * pNimi,
    TIEDOT * pAlku )

```

Parameters

<i>pNimi</i>	Kirjoitettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

void

Definition at line 164 of file linkitettylista.c.

```

164                                     {
165     FILE *Tiedosto = NULL;
166     TIEDOT *ptr = pAlku;
167
168     if (pAlku == NULL) {
169         printf("Lista on tyhjä, lue tiedosto ennen kirjoittamista.\n");
170         return;
171     }
172
173     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
174         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
175         exit(0);
176     }
177
178     // Mennään listan loppuun.
179     while (ptr->pSeuraava != NULL) {
180         ptr = ptr->pSeuraava;
181     }
182
183     // Lopusta alkuun, kirjoitetaan tiedostoon.
184     while (ptr != NULL) {
185         fprintf(Tiedosto, "%s,%d\n", ptr->aSukunimi, ptr->iYhteensa);
186         ptr = ptr->pEdellinen;
187     }
188
189     fclose(Tiedosto);
190     printf("Tiedosto %s kirjoitettu.\n", pNimi);
191     return;
192 }

```

4.6.2.4 laskeListanPituus()

```

int laskeListanPituus (
    TIEDOT * pAlku )

```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
--------------	-----------------------------------

Returns

int Linkitetyn listan alkioden lukumäärä.

Definition at line 362 of file linkitettylista.c.

```

362                                     {
363     int i = 0;
364     while (pAlku != NULL) {
365         i++;
366         pAlku = pAlku->pSeuraava;
367     }
368     return i;
369 }
```

4.6.2.5 lisääListaan()

```

TIEDOT* lisääListaan (
    TIEDOT * pAlku,
    int iIndeksi,
    char * pNimi,
    int iArvo )
```

Lisaa käyttäjän haluamaan kohtaan (indeksi) tiedot lisattavasta nimestä ja niiden lukumaarasta.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iIndeksi</i>	Käyttäjän antama indeksi, eli kohta, johon tietue lisataan linkitettyssä listassa. Jos lista on tyhjä, lisataan automaattisesti ensimmäiseksi elementiksi. Jos indeksi on suurempi kuin listan elementtien määrä, lisataan automaattisesti viimeiseksi.
<i>pNimi</i>	Lisattava nimi.
<i>iArvo</i>	Lisattavan nimen lukumaara.

Returns

pAlku Osoitin listan nykyiseen alkuun.

Definition at line 207 of file linkitettylista.c.

```

207                                     {
208     TIEDOT *pUusi = NULL;
209     TIEDOT *ptr = pAlku;
210     int i = 0;
211
212     // Varataan muistia uudelle linkitetyn listan solmulle.
213     if ((pUusi = (TIEDOT *)malloc(sizeof(TIEDOT))) == NULL) {
214         perror("Muistin varaus epäonnistui, lopetetaan");
215         exit(0);
216     }
217
218     // Lisataan tiedot tietueeseen.
219     strcpy(pUusi->aSukunimi, pNimi);
220     pUusi->iYhteensa = iArvo;
221 }
```

```

222 // Jos lista on tyhjä, lisataan uusi solmu ensimmäiseksi.
223 if (pAlku == NULL) {
224     pUusi->pEdellinen = NULL;
225     pUusi->pSeuraava = NULL;
226     pAlku = pUusi;
227     return (pAlku);
228 } else {
229     // Jos lisataan listan alkuun, eli indeksi = 0.
230     if (iIndeksi == 0) {
231         pUusi->pEdellinen = NULL;
232         pUusi->pSeuraava = pAlku;
233         pAlku->pEdellinen = pUusi;
234         pAlku = pUusi;
235         return (pAlku);
236     }
237
238     // Etsitään listalta oikea kohta, johon tiedot lisataan.
239     while (ptr->pSeuraava != NULL && i < (iIndeksi - 1)) {
240         ptr = ptr->pSeuraava;
241         i++;
242     }
243     // Lisataan tiedot oikeaan kohtaan linkitettyä listaa.
244     // Paivitetään osoittimet osoittamaan oikeisiin kohtiin.
245     pUusi->pSeuraava = ptr->pSeuraava;
246     pUusi->pEdellinen = ptr;
247
248     if (ptr->pSeuraava != NULL) {
249         ptr->pSeuraava->pEdellinen = pUusi;
250     }
251
252     ptr->pSeuraava = pUusi;
253 }
254 return (pAlku);
255 }

```

4.6.2.6 lomitus()

```

TIEDOT* lomitus (
    TIEDOT * p1,
    TIEDOT * p2 )

```

Parameters

<i>p1</i>	Osoitin ensimmäisen lajitellun listan alkuun.
<i>p2</i>	Osoitin jälkimmäisen lajitellun listan alkuun.

Returns

TIEDOT* yhdistetty lajiteltu lista.

Definition at line 409 of file linkitettylista.c.

```

409 {
410     // Jos kumpikaan lista on tyhjä, palautetaan toinen.
411     if (p1 == NULL)
412         return p2;
413     if (p2 == NULL)
414         return p1;
415
416     TIEDOT *p3 = NULL; // Uuden listan alku.
417     TIEDOT *p3Loppu = NULL; // Uuden lista loppu, jotta loppuun lisääminen onnistuu helposti.
418
419     // Käydään molempia listoja läpi:
420     while (p1 != NULL && p2 != NULL) {
421         if (p1->iYhteensa > p2->iYhteensa) {
422             TIEDOT *ptr = p2; // p2[0] talteen
423             // Poistetaan p2[0] listasta.
424             p2 = p2->pSeuraava;
425             if (p2 != NULL) {
426                 p2->pEdellinen = NULL;

```



```

427         }
428         // Lisätään p2[0] uuden listan loppuun.
429         ptr->pSeuraava = NULL;
430         ptr->pEdellinen = p3Loppu;
431         if (p3Loppu != NULL) {
432             p3Loppu->pSeuraava = ptr;
433         } else {
434             p3 = ptr; // ensimmäinen alkio, jos lista tyhjä
435         }
436         p3Loppu = ptr; // päivitetään loppu.
437     } else {
438         TIEDOT *ptr = p1; // p1[0] talteen
439         p1 = p1->pSeuraava;
440         // Poistetaan p1[0] listasta.
441         if (p1 != NULL) {
442             p1->pEdellinen = NULL;
443         }
444         // Lisätään p1[0] uuden listan loppuun.
445         ptr->pSeuraava = NULL;
446         ptr->pEdellinen = p3Loppu;
447         if (p3Loppu != NULL) {
448             p3Loppu->pSeuraava = ptr;
449         } else {
450             p3 = ptr; // ensimmäinen alkio, jos lista tyhjä
451         }
452         p3Loppu = ptr; // päivitetään loppu.
453     }
454 }
455
456 // Tässä kohtaa joko p1 tai p2 on tyhjä.
457
458 // Lisätään loput p1:stä uuden listan loppuun.
459 while (p1 != NULL) {
460     TIEDOT *ptr = p1;
461     p1 = p1->pSeuraava;
462     if (p1 != NULL) {
463         p1->pEdellinen = NULL;
464     }
465     ptr->pSeuraava = NULL;
466     ptr->pEdellinen = p3Loppu;
467     if (p3Loppu != NULL) {
468         p3Loppu->pSeuraava = ptr;
469     } else {
470         p3 = ptr;
471     }
472     p3Loppu = ptr;
473 }
474
475 // Lisätään loput p2:stä uuden listan loppuun.
476 while (p2 != NULL) {
477     TIEDOT *ptr = p2;
478     p2 = p2->pSeuraava;
479     if (p2 != NULL) {
480         p2->pEdellinen = NULL;
481     }
482     ptr->pSeuraava = NULL;
483     ptr->pEdellinen = p3Loppu;
484     if (p3Loppu != NULL) {
485         p3Loppu->pSeuraava = ptr;
486     } else {
487         p3 = ptr;
488     }
489     p3Loppu = ptr;
490 }
491 return p3;
492 }

```

4.6.2.7 lomitusLajittelu()

```

TIEDOT* lomitusLajittelu (
    TIEDOT * pAlku )

```

Parameters

<i>pAlku</i>	Osoitin lajiteltavan linkitetyn listan alkuun.
--------------	--

Returns

TIEDOT* Lajiteltu lista.

Definition at line 500 of file linkitettylista.c.

```

500                                     {
501     TIEDOT *p1 = NULL, *p2 = NULL;
502
503     // Jos lista on tyhjä tai sisältää vain yhden alkion
504     if (pAlku == NULL || pAlku->pSeuraava == NULL) {
505         return pAlku;
506     }
507
508     // Jaetaan lista kahtia
509     p2 = halkaise(pAlku);
510     p1 = pAlku;
511
512     // Lajitellaan molemmat puolikkaat
513     p1 = lomituspLajittelu(p1);
514     p2 = lomituspLajittelu(p2);
515
516     // Yhdistetään lajitellut puolikkaat
517     return lomitusp(p1, p2);
518 }
```

4.6.2.8 lue()

```

TIEDOT* lue (
    char * pNimi,
    TIEDOT * pAlku )
```

Avaa ja lukee käyttäjän syötteen mukaisen tiedoston. Kutsuu muistinvaraus aliohjelman muistin varaamiseksi.

Parameters

<i>pNimi</i>	Luettavan tiedoston nimi.
<i>pAlku</i>	Osoitin linkitetyn listan alkuun.

Returns

pAlku Uusi osoitin linkitetyn listan alkuun.

Definition at line 18 of file linkitettylista.c.

```

18                                     {
19     FILE *Tiedosto = NULL;
20     char aRivi[LEN] = "";
21     char *p1 = NULL, *p2 = NULL;
22     int iOtsikko = 0;
23     int iMaara = 0;
24
25     /* Tiedoston avaaminen. */
26
27     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
28         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
29         exit(0);
30     }
31
32     /* Tiedoston lukeminen */
33
34     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
35
36         aRivi[strlen(aRivi) - 1] = '\0';
37
38         if (iOtsikko == 0) {
39             iOtsikko = 1;
40             continue;
```

```

41     }
42
43     if ((p1 = strtok(aRivi, ";")) == NULL) {
44         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
45         exit(0);
46     }
47
48     if ((p2 = strtok(NULL, ";")) == NULL) {
49         perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
50         exit(0);
51     }
52
53     iMaara = atoi(p2);
54
55     /* Muistin varaaminen ja linkitetyn listan luominen. */
56
57     pAlku = varaaMuistia(pAlku, p1, iMaara);
58 }
59 /* Tiedoston sulkeminen. */
60 fclose(Tiedosto);
61 printf("Tiedosto '%s' luettu.\n", pNimi);
62 return (pAlku);
63 }

```

4.6.2.9 poistaListastaLkmPeruusteella()

```

TIEDOT* poistaListastaLkmPeruusteella (
    TIEDOT * pAlku,
    int iLKM )

```

Käyttäjä on antanut lukumäärän. Alkio, jossa on tämä lukumäärä poistetaan.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iLKM</i>	Käyttäjän antama lukumäärä. Alkio, jossa on tämä lukumäärä poistetaan.

Returns

TIEDOT* Palautetaan linkitettylista, josta on poistettu alkio.

Definition at line 264 of file linkitettylista.c.

```

264
265     TIEDOT *ptr = pAlku;
266     TIEDOT *pEdellinen = NULL;
267
268     // Tarkistetaan, onko poistettava alkio ensimmäinen.
269     if ((ptr != NULL) && (ptr->iYhteensa == iLKM)) {
270         // Sijoitetaan osoitin osoittamaan seuraavaan alkioon.
271         pAlku = ptr->pSeuraava;
272         free(ptr);
273         printf("Alkio poistettu.\n");
274
275         // Jos poistettava alkio ei ole ensimmäinen.
276     } else {
277         // Etsitään kohtaa, jossa poistettava alkio on.
278         while (ptr != NULL) {
279             if (ptr->iYhteensa == iLKM) {
280                 pEdellinen->pSeuraava = ptr->pSeuraava;
281                 free(ptr);
282                 printf("Alkio poistettu.\n");
283                 break;
284             }
285             pEdellinen = ptr;
286             ptr = ptr->pSeuraava;
287         }
288     }
289

```

```

290     return (pAlku);
291 }

```

4.6.2.10 poistaListastaNimenPerusteella()

```

TIEDOT* poistaListastaNimenPerusteella (
    TIEDOT * pAlku,
    char * pNimi )

```

Käyttäjä on antanut nimen. Alkio, jossa on tämä nimi poistetaan.

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>pNimi</i>	Käyttäjän antama nimi. Alkio, jossa on tämä nimi poistetaan.

Returns

TIEDOT* TIEDOT* Palautetaan linkitettylista, josta on poistettu alkio.

Definition at line 300 of file linkitettylista.c.

```

300
301     TIEDOT *ptr = pAlku;
302     TIEDOT *pEdellinen = NULL;
303
304     // Tarkistetaan, onko poistettava alkio ensimmäinen.
305     if ((ptr != NULL) && (strcmp(ptr->aSukunimi, pNimi) == 0)) {
306         // Sijoitetaan osoitin osoittamaan seuraavaan alkioon.
307         pAlku = ptr->pSeuraava;
308         free(ptr);
309         printf("Alkio poistettu.\n");
310
311         // Jos poistettava alkio ei ole ensimmäinen.
312     } else {
313         // Etsitään kohtaa, jossa poistettava alkio on.
314         while (ptr != NULL) {
315             if (strcmp(ptr->aSukunimi, pNimi) == 0) {
316                 pEdellinen->pSeuraava = ptr->pSeuraava;
317                 free(ptr);
318                 printf("Alkio poistettu.\n");
319                 break;
320             }
321             pEdellinen = ptr;
322             ptr = ptr->pSeuraava;
323         }
324     }
325     return (pAlku);
326 }

```

4.6.2.11 useammallaAlkiollaSamaLKM()

```

int useammallaAlkiollaSamaLKM (
    TIEDOT * pAlku,
    int iLKM )

```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>iLKM</i>	Käyttäjän antama lukumäärä, joiden määrä selvitetään.

Returns

int Palauttaa luvun, jossa on kaikki alkiot, joissa on käyttäjän antama luku.

Definition at line 335 of file linkitettylista.c.

```
335                                     {
336     int iSamoja = 0;
337     TIEDOT *ptr = NULL;
338     ptr = pAlku;
339
340     // Käydään linkitettylista läpi
341     while (ptr != NULL) {
342         // Lisätään summaan yksi, jos linkitetyn listan alkion lukumäärä on sama kuin käyttäjän
343         // antama luku.
344         if (ptr->iYhteensa == iLKM) {
345             iSamoja++;
346         }
347         ptr = ptr->pSeuraava;
348     }
349
350     return (iSamoja);
351 }
```

4.6.2.12 vapautaMuisti()

```
TIEDOT* vapautaMuisti (
    TIEDOT * pAlku )
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
--------------	-----------------------------------

Returns

pAlku Osoitin, joka on NULL, koska lista on tyhja.

Definition at line 110 of file linkitettylista.c.

```
110                                     {
111     /* Muistin vapauttaminen. */
112     TIEDOT *ptr = pAlku;
113     while (ptr != NULL) {
114         pAlku = ptr->pSeuraava;
115         free(ptr);
116         ptr = pAlku;
117     }
118     pAlku = NULL;
119     return (pAlku);
120 }
```

4.6.2.13 varaaMuistia()

```
TIEDOT* varaaMuistia (
    TIEDOT * pAlku,
    char * pNimi,
    int iMaara )
```

Parameters

<i>pAlku</i>	Osoitin linkitetyn listan alkuun.
<i>pNimi</i>	Merkkijono solmun nimeksi.
<i>iMaara</i>	Luku, joka asetetaan solmuun maaraksi.

Returns

pAlku Uusi osoitin linkitetyn listan alkuun.

Definition at line 73 of file linkitettylista.c.

```

73                                     {
74     TIEDOT *pUusi = NULL;
75     TIEDOT *ptr = NULL;
76
77     /* Muistin varaaminen. */
78
79     if ((pUusi = (TIEDOT *)malloc(sizeof(TIEDOT))) == NULL) {
80         perror("Muistin varaus epäonnistui, lopetetaan");
81         exit(0);
82     }
83
84     /* Kaksisuuntaisen linkitetyn listan luominen. */
85
86     strcpy(pUusi->aSukunimi, pNimi);
87     pUusi->iYhteensa = iMaara;
88     pUusi->pSeuraava = NULL;
89
90     if (pAlku == NULL) {
91         pUusi->pEdellinen = NULL;
92         pAlku = pUusi;
93     } else {
94         ptr = pAlku;
95         while (ptr->pSeuraava != NULL) {
96             ptr = ptr->pSeuraava;
97         }
98         ptr->pSeuraava = pUusi;
99         pUusi->pEdellinen = ptr;
100     }
101     return (pAlku);
102 }
```

4.7 paaohjelma.c File Reference

```

#include "binaaripuu.h"
#include "linkitettylista.h"
#include "yleiset.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- int [main](#) (void)
Paaohjelma, pyorittaa koko ohjelmaa.

4.7.1 Function Documentation

4.7.1.1 main()

```
int main (
    void )
```

Returns

int Palauttaa aina 0, kun ohjelma tulee päätökseen.

Definition at line 13 of file paaohjelma.c.

```
13     {
14     int iValinta = 0;
15
16     do {
17         iValinta = valikko();
18         if (iValinta == 1) {
19             mainLista();
20         } else if (iValinta == 2) {
21             mainBinaaripuu();
22         } else if (iValinta == 0) {
23             printf("Lopetetaan.\n");
24         } else {
25             printf("Tuntematon valinta, yritä uudestaan.\n");
26         }
27         printf("\n");
28     } while (iValinta != 0);
29
30     printf("Kiitos ohjelman käytöstä.\n");
31     return (0);
32 }
```

4.8 punamustapuu.c File Reference

```
#include "binaaripuu.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- [RBSOLMU * varaaMuistiaRB](#) (char *pNimi, int iArvo)
L10 / Punamustapuu Vaatii oman structin takia omat aliohjelmat, jotka ovat [binaaripuu.c](#) tiedoston aliohjelmia muokattuina.
- [RBSOLMU * vapautaMuistiRB](#) (RBSOLMU *pJuuriSolmu)
- [RBSOLMU * lisaaRBSolmu](#) (RBSOLMU *pJuuriSolmu, RBSOLMU *pUusi)
- [RBSOLMU * luoRBPuu](#) (RBSOLMU *pJuuriSolmu, char *pNimi)
- void [kierraVasemmalle](#) (RBSOLMU **pJuuriSolmu, RBSOLMU *pSolmu)
- void [kierraOikealle](#) (RBSOLMU **pJuuriSolmu, RBSOLMU *pSolmu)
- void [korjaaLisays](#) (RBSOLMU **pJuuriSolmu, RBSOLMU *pUusi)
- void [kirjoitaRBTiedostoon](#) (char *pNimi, RBSOLMU *pJuuriSolmu)
- void [kirjoitaRB](#) (char *pNimi, RBSOLMU *pJuuriSolmu)

4.8.1 Function Documentation

4.8.1.1 kierraOikealle()

```
void kierraOikealle (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pSolmu )
```

Definition at line 130 of file punamustapuu.c.

```
130 {
131     RBSOLMU *pVasenLapsi = pSolmu->pVasen;
132     pSolmu->pVasen = pVasenLapsi->pOikea;
133
134     if (pSolmu->pVasen != NULL)
135         pSolmu->pVasen->pVanhempi = pSolmu;
136
137     pVasenLapsi->pVanhempi = pSolmu->pVanhempi;
138
139     if (pSolmu->pVanhempi == NULL)
140         *pJuurisolmu = pVasenLapsi;
141     else if (pSolmu == pSolmu->pVanhempi->pVasen)
142         pSolmu->pVanhempi->pVasen = pVasenLapsi;
143     else
144         pSolmu->pVanhempi->pOikea = pVasenLapsi;
145
146     pVasenLapsi->pOikea = pSolmu;
147     pSolmu->pVanhempi = pVasenLapsi;
148 }
```

4.8.1.2 kierraVasemmalle()

```
void kierraVasemmalle (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pSolmu )
```

Definition at line 109 of file punamustapuu.c.

```
109 {
110     RBSOLMU *pOikeaLapsi = pSolmu->pOikea;
111     pSolmu->pOikea = pOikeaLapsi->pVasen;
112
113     if (pSolmu->pOikea != NULL)
114         pSolmu->pOikea->pVanhempi = pSolmu;
115
116     // Solmun oikea lapsi nousee ylös puussa
117     pOikeaLapsi->pVanhempi = pSolmu->pVanhempi;
118
119     if (pSolmu->pVanhempi == NULL)
120         *pJuurisolmu = pOikeaLapsi;
121     else if (pSolmu == pSolmu->pVanhempi->pVasen)
122         pSolmu->pVanhempi->pVasen = pOikeaLapsi;
123     else
124         pSolmu->pVanhempi->pOikea = pOikeaLapsi;
125
126     pOikeaLapsi->pVasen = pSolmu;
127     pSolmu->pVanhempi = pOikeaLapsi;
128 }
```

4.8.1.3 kirjoitaRB()

```
void kirjoitaRB (
    char * pNimi,
    RBSOLMU * pJuurisolmu )
```

Definition at line 222 of file punamustapuu.c.

```
222 {
223     /*if (pJuurisolmu == NULL) {
```



```

224     printf("Puu on tyhjä, luo puurakenne ennen kirjoittamista.\n");
225     return;
226 }*/
227 if (pJuurisolmu != NULL) {
228     kirjoitaRBTiedostoon(pNimi, pJuurisolmu);
229     kirjoitaRB(pNimi, pJuurisolmu->pVasen);
230     kirjoitaRB(pNimi, pJuurisolmu->pOikea);
231 }
232 return;
233 }

```

4.8.1.4 kirjoitaRBTiedostoon()

```

void kirjoitaRBTiedostoon (
    char * pNimi,
    RBSOLMU * pJuurisolmu )

```

Definition at line 201 of file punamustapuu.c.

```

201                                     {
202     FILE *Tiedosto = NULL;
203
204     if (pJuurisolmu == NULL) {
205         return;
206     }
207
208     /* Tiedoston avaaminen. */
209     if ((Tiedosto = fopen(pNimi, "a")) == NULL) {
210         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
211         exit(0);
212     }
213     /* Tiedostoon kirjoittaminen. */
214     fprintf(Tiedosto, "%s,%d\n", pJuurisolmu->aNimi, pJuurisolmu->iArvo);
215
216     /* Tiedoston sulkeminen. */
217     fclose(Tiedosto);
218     printf("Tiedosto '%s' kirjoitettu.\n", pNimi);
219     return;
220 }

```

4.8.1.5 korjaaLisays()

```

void korjaaLisays (
    RBSOLMU ** pJuurisolmu,
    RBSOLMU * pUusi )

```

Definition at line 150 of file punamustapuu.c.

```

150                                     {
151     // Ilman p-alkua, koska pVanhempi->pVanhempi oli sekava
152     RBSOLMU *vanhempi = NULL;
153     RBSOLMU *seta = NULL;
154     RBSOLMU *isoVanhempi = NULL;
155
156     while ((pUusi != *pJuurisolmu) && (pUusi->pVanhempi->iVariBitti == 0)) {
157         vanhempi = pUusi->pVanhempi;
158         isoVanhempi = vanhempi->pVanhempi;
159
160         if (vanhempi == isoVanhempi->pVasen) {
161             seta = isoVanhempi->pOikea;
162             if (seta != NULL && seta->iVariBitti == 0) {
163                 vanhempi->iVariBitti = 1;
164                 seta->iVariBitti = 1;
165                 isoVanhempi->iVariBitti = 0;
166                 pUusi = isoVanhempi;
167             } else {
168                 if (pUusi == vanhempi->pOikea) {
169                     pUusi = vanhempi;
170                     kierraVasemmalle(pJuurisolmu, pUusi);
171                     vanhempi = pUusi->pVanhempi;

```

```

172         }
173         vanhempi->iVariBitti = 1;
174         isoVanhempi->iVariBitti = 0;
175         kierraOikealle(pJuurisolmu, isoVanhempi);
176     }
177
178     } else {
179         seta = isoVanhempi->pVasen;
180         if (seta != NULL && seta->iVariBitti == 0) {
181             vanhempi->iVariBitti = 1;
182             seta->iVariBitti = 1;
183             isoVanhempi->iVariBitti = 0;
184             pUusi = isoVanhempi;
185         } else {
186             if (pUusi == vanhempi->pVasen) {
187                 pUusi = vanhempi;
188                 kierraOikealle(pJuurisolmu, pUusi);
189                 vanhempi = pUusi->pVanhempi;
190             }
191             vanhempi->iVariBitti = 1;
192             isoVanhempi->iVariBitti = 0;
193             kierraVasemmalle(pJuurisolmu, isoVanhempi);
194         }
195     }
196 }
197 (*pJuurisolmu)->iVariBitti = 1;
198 }

```

4.8.1.6 lisaaRBSolmu()

```

RBSOLMU* lisaaRBSolmu (
    RBSOLMU * pJuurisolmu,
    RBSOLMU * pUusi )

```

Katsotaan, ovatko arvot samat. Laitetaan aakkosten perusteella vasemmalle tai oikealle.

Definition at line 38 of file punamustapuu.c.

```

38                                     {
39     int iVertailu = 0;
40
41     // Jos puu on tyhjä, palautetaan uusi RBSolmu.
42     if (pJuurisolmu == NULL) {
43         return pUusi;
44     }
45
46     iVertailu = strcmp(pUusi->aNimi, pJuurisolmu->aNimi);
47
48     // Solmujen lisääminen rekursiivisesti
49     if (pUusi->iArvo < pJuurisolmu->iArvo) {
50         pJuurisolmu->pVasen = lisaaRBSolmu(pJuurisolmu->pVasen, pUusi);
51         pJuurisolmu->pVasen->pVanhempi = pJuurisolmu;
52     } else if (pUusi->iArvo > pJuurisolmu->iArvo) {
53         pJuurisolmu->pOikea = lisaaRBSolmu(pJuurisolmu->pOikea, pUusi);
54         pJuurisolmu->pOikea->pVanhempi = pJuurisolmu;
55     } else if (pUusi->iArvo == pJuurisolmu->iArvo) {
56         if (iVertailu < 0) {
57             pJuurisolmu->pVasen = lisaaRBSolmu(pJuurisolmu->pVasen, pUusi);
58             pJuurisolmu->pVasen->pVanhempi = pJuurisolmu;
59         } else if (iVertailu > 0) {
60             pJuurisolmu->pOikea = lisaaRBSolmu(pJuurisolmu->pOikea, pUusi);
61             pJuurisolmu->pOikea->pVanhempi = pJuurisolmu;
62         }
63     }
64     return (pJuurisolmu);
65 }
66
67 }

```

4.8.1.7 luoRBPuu()

```
RBSOLMU* luoRBPuu (
    RBSOLMU * pJuurisolmu,
    char * pNimi )
```

Definition at line 69 of file punamustapuu.c.

```
69 {
70     FILE *Tiedosto = NULL;
71     char aRivi[LEN] = "";
72     int iOtsikko = 0;
73     char *p1 = NULL, *p2 = NULL;
74     int iArvo = 0;
75
76     if ((Tiedosto = fopen(pNimi, "r")) == NULL) {
77         perror("Tiedoston avaaminen epäonnistui, lopetetaan");
78         exit(0);
79     }
80
81     while ((fgets(aRivi, LEN, Tiedosto)) != NULL) {
82         aRivi[strlen(aRivi) - 1] = '\\0';
83         if (iOtsikko == 0) {
84             iOtsikko++;
85             continue;
86         }
87
88         if ((p1 = strtok(aRivi, ";")) == NULL) {
89             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
90             exit(0);
91         }
92
93         if ((p2 = strtok(NULL, ";")) == NULL) {
94             perror("Tiedoston pilkkominen epäonnistui, lopetetaan");
95             exit(0);
96         }
97
98         iArvo = atoi(p2);
99
100         RBSOLMU *pUusi = varaaMuistiaRB(p1, iArvo);
101         pJuurisolmu = lisaaRBSolmu(pJuurisolmu, pUusi);
102         korjaaLisays(&pJuurisolmu, pUusi);
103     }
104
105     fclose(Tiedosto);
106     return (pJuurisolmu);
107 }
```

4.8.1.8 vapautaMuistiRB()

```
RBSOLMU* vapautaMuistiRB (
    RBSOLMU * pJuuriSolmu )
```

Definition at line 28 of file punamustapuu.c.

```
28 {
29     if (pJuuriSolmu != NULL) {
30         vapautaMuistiRB(pJuuriSolmu->pVasen);
31         vapautaMuistiRB(pJuuriSolmu->pOikea);
32         free(pJuuriSolmu);
33     }
34     pJuuriSolmu = NULL;
35     return (pJuuriSolmu);
36 }
```

4.8.1.9 varaaMuistiaRB()

```
RBSOLMU* varaaMuistiaRB (
    char * pNimi,
    int iArvo )
```

Punamustapuu, ehkä oma header olisi kätevämpi, kun omat aliohjelmat oman structin takia.

Definition at line 11 of file punamustapuu.c.

```
11                                     {
12     RBSOLMU *pUusi = NULL;
13
14     if ((pUusi = (RBSOLMU *)malloc(sizeof(RBSOLMU))) == NULL) {
15         perror("Muistin varaus epäonnistui, lopetetaan");
16         exit(0);
17     }
18
19     pUusi->iVariBitti = 0; // uudet aina punaisia
20     strcpy(pUusi->aNimi, pNimi);
21     pUusi->iArvo = iArvo;
22     pUusi->pVasen = NULL;
23     pUusi->pOikea = NULL;
24     pUusi->pVanhempi = NULL;
25     return (pUusi);
26 }
```

4.9 yleiset.c File Reference

```
#include "yleiset.h"
#include "binaaripuu.h"
#include "haut.h"
#include "linkitettylista.h"
#include <stdio.h>
#include <stdlib.h>
#include <string.h>
```

Functions

- void [mainLista](#) (void)
Tekee käyttäjän valinnan perusteella listan toiminnot.
- void [mainBinaaripuu](#) (void)
Tekee käyttäjän valinnan perusteella binääripuun toiminnot.
- int [valikko](#) (void)
Ensimmäinen valikko, jossa käyttäjä valitsee listan tai binaaripuun.
- int [listaValikko](#) (void)
Tulostaa lista valikon ja kysyy käyttäjän valinnan.
- int [binaaripuuValikko](#) (void)
Tulostaa binaariluku valikon ja kysyy käyttäjältä valinnan.
- char * [kysyNimi](#) (char *pPrompti, char *pNimi)
Kysyy tiedoston/haettavan nimen.
- int [kysyArvo](#) (char *pPrompti, int iArvo)
Kysyy arvon, joka halutaan etsiä/poistaa puusta.

4.9.1 Function Documentation

4.9.1.1 binaaripuuValikko()

```
int binaaripuuValikko (
    void )
```

Parameters

<i>iValinta</i>	Kayttajan antama syote vaihtoehtojen pohjalta.
-----------------	--

Returns

int Palauttaa kayttajan valinnan.

Definition at line 233 of file yleiset.c.

```
233     {
234         int iValinta = 0;
235         printf("\nValitse haluamasi toiminto:\n");
236         printf("1) Luo puu\n");
237         printf("2) Kirjoita AVL puu tiedostoon\n");
238         printf("3) Syvyyshaku\n");
239         printf("4) Leveyshaku\n");
240         printf("5) Tyhjennä puu\n");
241         printf("6) Poista solmu\n");
242         printf("7) Binäärihaku AVL puusta\n");
243         printf("8) Punamustapuu\n");
244         printf("0) Takaisin\n");
245         printf("Anna valintasi: ");
246         scanf("%d", &iValinta);
247         getchar();
248         return iValinta;
249     }
```

4.9.1.2 kysyArvo()

```
int kysyArvo (
    char * pPrompti,
    int iArvo )
```

Parameters

<i>pPrompti</i>	Kysymys esim. "Anna haettava numeroarvo: "
<i>iArvo</i>	Arvo, joka halutaan poistaa/etsiä.

Returns

int Palauttaa kayttajan antaman arvon.

Definition at line 272 of file yleiset.c.

```
272     {
273         printf("%s", pPrompti);
274         scanf("%d", &iArvo);
275         getchar();
276         return (iArvo);
277     }
```

4.9.1.3 kysyNimi()

```
char* kysyNimi (
    char * pPrompti,
    char * pNimi )
```

Parameters

<i>pPrompti</i>	Kysymys esim. "Anna kirjoitettavan tiedoston nimi: "
<i>pNimi</i>	Tiedoston nimi/haettava nimi, jota halutaan kayttaa.

Returns

char Palauttaa kayttajan antaman merkkijonon.

Definition at line 258 of file yleiset.c.

```
258                                     {
259     printf("%s", pPrompti);
260     scanf("%s", pNimi);
261     getchar();
262     return (pNimi);
263 }
```

4.9.1.4 listaValikko()

```
int listaValikko (
    void )
```

Parameters

<i>iValinta</i>	Kayttajan antama syote vaihtoehtojen pohjalta.
-----------------	--

Returns

int Palauttaa kayttajan valinnan.

Definition at line 209 of file yleiset.c.

```
209     {
210     int iValinta = 0;
211     printf("\nValitse haluamasi toiminto:\n");
212     printf("1) Lue tiedosto\n");
213     printf("2) Kirjoita tiedosto alusta loppuun\n");
214     printf("3) Kirjoita tiedosto lopusta alkuun\n");
215     printf("4) Tyhjennä taulukko\n");
216     printf("5) Lajittele nousevasti\n");
217     printf("6) Lajittele laskevasti\n");
218     printf("7) Lisää tietoja listaan\n");
219     printf("8) Poista listalta tietoja\n");
220     printf("0) Takaisin\n");
221     printf("Anna valintasi: ");
222     scanf("%d", &iValinta);
223     getchar();
224     return iValinta;
225 }
```

4.9.1.5 mainBinaaripuu()

```
void mainBinaaripuu (
    void )
```

Funktio näyttää valikon ja suorittaa käyttäjän valinnan mukaiset binääripuuhun liittyvät toiminnot.

Definition at line 97 of file yleiset.c.

```
97     {
98     char aNimiLuettava[LEN] = "";
99     char aNimiKirjoitettava[LEN] = "";
100     char aHaettavaNimi[LEN] = "";
101     PUU *pJuuriSolmu = NULL;
102     RBSOLMU *pRBJuuri = NULL;
103     int iValinta = 0;
104     int iArvo = 0;
105
106     do {
107         iValinta = binaaripuuValikko();
108         if (iValinta == 1) {
109             kysyNimi("Tiedoston nimi: ", aNimiLuettava);
110             pJuuriSolmu = luoPuu(aNimiLuettava, pJuuriSolmu);
111         } else if (iValinta == 2) {
112             if (pJuuriSolmu == NULL) {
113                 printf("Luo binääripuu ennen sen kirjoittamista.\n");
114             } else {
115                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
116                 kirjoitaBinaaripuu(aNimiKirjoitettava, pJuuriSolmu);
117             }
118         } else if (iValinta == 3) {
119             if (pJuuriSolmu == NULL) {
120                 printf("Luo binääripuu ennen syvyyshakua.\n");
121             } else {
122                 iArvo = kysyArvo("Anna haettava numeroarvo: ", iArvo);
123                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
124                 tarkistaLoytyykoSyvyyshaulla(aNimiKirjoitettava, pJuuriSolmu, iArvo);
125             }
126         } else if (iValinta == 4) {
127             if (pJuuriSolmu == NULL) {
128                 printf("Luo binääripuu ennen leveyshakua.\n");
129             } else {
130                 kysyNimi("Anna haettava nimi: ", aHaettavaNimi);
131                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
132                 leveyshaku(aNimiKirjoitettava, pJuuriSolmu, aHaettavaNimi);
133             }
134         } else if (iValinta == 5) {
135             pJuuriSolmu = vapautaMuistiPuu(pJuuriSolmu);
136             printf("Muisti vapautettu.\n");
137         } else if (iValinta == 6) {
138             if (pJuuriSolmu == NULL) {
139                 printf("Luo binääripuu ennen poistoa.\n");
140             } else {
141                 kysyNimi("Anna poistettava nimi tai arvo: ", aHaettavaNimi);
142                 pJuuriSolmu = poistaSolmu(aHaettavaNimi, pJuuriSolmu);
143             }
144         } else if (iValinta == 7) {
145             if (pJuuriSolmu == NULL) {
146                 printf("Luo binääripuu ennen binäärihakua.\n");
147             } else {
148                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
149                 iArvo = kysyArvo("Anna haettava numeroarvo: ", iArvo);
150                 iArvo = binaarihaku(aNimiKirjoitettava, pJuuriSolmu, iArvo);
151                 if (iArvo == 0) {
152                     printf("Hakemaasi arvoa ei löytynyt binääripuusta.\n");
153                 }
154             }
155         } else if (iValinta == 8) {
156             // Aino säätää ja testaa
157             kysyNimi("Anna luettavan tiedoston nimi: ", aNimiLuettava);
158             kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
159             pRBJuuri = luoRBPuu(pRBJuuri, aNimiLuettava);
160             kirjoitaRB(aNimiKirjoitettava, pRBJuuri);
161             pRBJuuri = vapautaMuistiRB(pRBJuuri);
162         } else if (iValinta == 0) {
163             printf("Palataan takaisin päävalikkoon.\n");
164         }
165     } while (iValinta != 0);
166 }
```

```

172
173     } else {
174         printf("Tuntematon valinta, yritä uudestaan.\n");
175     }
176 } while (iValinta != 0);
177
178 /* Vapautetaan puun varaama muisti varmuuden vuoksi vielä täällä lopussa erikseen, jos
179  * käyttäjä ei muista sitä tehdä. */
180
181 pJuuriSolmu = vapautaMuistiPuu(pJuuriSolmu);
182 return;
183 }

```

4.9.1.6 mainLista()

```

void mainLista (
    void )

```

Funktio näyttää valikon ja suorittaa käyttäjän valinnan mukaiset listaan liittyvät toiminnot.

Definition at line 17 of file yleiset.c.

```

17     {
18         char aNimiLuettava[LEN] = "";
19         char aNimiKirjoitettava[LEN] = "";
20         int iArvo = 0;
21         int iIndeksi = 0;
22         int iValinta = 0;
23         int iSamaLKM = 0;
24         TIEDOT *pAlku = NULL;
25
26         do {
27             iValinta = listaValikko();
28             if (iValinta == 1) {
29                 kysyNimi("Anna luettavan tiedoston nimi: ", aNimiLuettava);
30                 pAlku = lue(aNimiLuettava, pAlku);
31
32             } else if (iValinta == 2) {
33                 if (pAlku == NULL) {
34                     printf("Lue tiedosto ennen kirjoittamista.\n");
35                 } else {
36                     kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
37                     kirjoitaTiedostoAlustaLoppuun(aNimiKirjoitettava, pAlku);
38                 }
39
40             } else if (iValinta == 3) {
41                 if (pAlku == NULL) {
42                     printf("Lue tiedosto ennen kirjoittamista.\n");
43                 } else {
44                     kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
45                     kirjoitaTiedostoLopustaAlkuun(aNimiKirjoitettava, pAlku);
46                 }
47
48             } else if (iValinta == 4) {
49                 pAlku = vapautaMuisti(pAlku);
50                 printf("Muisti vapautettu.\n");
51
52             } else if (iValinta == 5) {
53                 pAlku = lomitusLajittelu(pAlku);
54                 printf("Numerot lajiteltu nousevasti.\n");
55
56             } else if (iValinta == 6) {
57                 // Laskeva lajittelu
58
59             } else if (iValinta == 7) {
60                 iIndeksi = kysyArvo("Mihin kohtaan listaa haluat lisätä tietoja (indeksi): ", iIndeksi);
61                 kysyNimi("Anna lisättävä nimi: ", aNimiKirjoitettava);
62                 iArvo = kysyArvo("Anna tämän nimisten lukumäärä: ", iArvo);
63                 pAlku = lisaaListaan(pAlku, iIndeksi, aNimiKirjoitettava, iArvo);
64
65             } else if (iValinta == 8) {
66                 // Poistaminen listalta
67                 iArvo = kysyArvo("Anna poistettava lukumäärä: ", iArvo);
68                 iSamaLKM = useammallaAlkiollaSamaLKM(pAlku, iArvo);
69                 if (iSamaLKM == 1) {
70                     pAlku = poistaListastaLkmPeruusteella(pAlku, iArvo);
71                 } else if (iSamaLKM > 1) {
72                     kysyNimi("Anna poistettava nimi: ", aNimiKirjoitettava);

```



```

73         pAlku = poistaListastaNimenPerusteella(pAlku, aNimiKirjoitettava);
74     }
75
76     } else if (iValinta == 0) {
77         printf("Palataan takaisin päävalikkoon.\n");
78
79     } else {
80         printf("Tuntematon valinta, yritä uudestaan.\n");
81     }
82     } while (iValinta != 0);
83
84     /* Vapautetaan listan varaama muisti varmuuden vuoksi vielä täällä lopussa erikseen, jos
85      * käyttäjä ei muista sitä tehdä. */
86
87     pAlku = vapautaMuisti(pAlku);
88     return;
89 }

```

4.9.1.7 valikko()

```

int valikko (
    void )

```

Parameters

<i>iValinta</i>	Kayttajan antama syote.
-----------------	-------------------------

Returns

int Palauttaa kayttajan valinnan.

Definition at line 191 of file yleiset.c.

```

191     {
192         int iValinta = 0;
193         printf("Valitse haluamasi toiminto:\n");
194         printf("1) Lista\n");
195         printf("2) Binääripuu\n");
196         printf("0) Lopeta\n");
197         printf("Anna valintasi: ");
198         scanf("%d", &iValinta);
199         getchar();
200         return iValinta;
201     }

```

4.10 yleiset.h File Reference

Macros

- #define [LEN](#) 50

Functions

- void [mainLista](#) (void)
Tekee käyttäjän valinnan perusteella listan toiminnot.
- void [mainBinaaripuu](#) (void)
Tekee käyttäjän valinnan perusteella binääripuun toiminnot.
- int [valikko](#) (void)

Ensimmäinen valikko, jossa käyttäjä valitsee listan tai binaaripuun.

- int `listaValikko` (void)
Tulostaa lista valikon ja kysyy käyttäjän valinnan.
- int `binaaripuuValikko` (void)
Tulostaa binaariluku valikon ja kysyy käyttäjältä valinnan.
- char * `kysyNimi` (char *pPrompti, char *pNimi)
Kysyy tiedoston/haettavan nimen.
- int `kysyArvo` (char *pPrompti, int iArvo)
Kysyy arvon, joka halutaan etsiä/poistaa puusta.

4.10.1 Macro Definition Documentation

4.10.1.1 LEN

```
#define LEN 50
```

Definition at line 4 of file yleiset.h.

4.10.2 Function Documentation

4.10.2.1 binaaripuuValikko()

```
int binaaripuuValikko (
    void )
```

Parameters

<i>iValinta</i>	Käyttäjän antama syöte vaihtoehtojen pohjalta.
-----------------	--

Returns

int Palauttaa käyttäjän valinnan.

Definition at line 233 of file yleiset.c.

```
233     {
234         int iValinta = 0;
235         printf("\nValitse haluamasi toiminto:\n");
236         printf("1) Luo puu\n");
237         printf("2) Kirjoita AVL puu tiedostoon\n");
238         printf("3) Syvyyshaku\n");
239         printf("4) Leveyshaku\n");
240         printf("5) Tyhjennä puu\n");
241         printf("6) Poista solmu\n");
242         printf("7) Binäärihaku AVL puusta\n");
243         printf("8) Punamustapuu\n");
244         printf("0) Takaisin\n");
245         printf("Anna valintasi: ");
```

```
246     scanf("%d", &iValinta);
247     getchar();
248     return iValinta;
249 }
```

4.10.2.2 kysyArvo()

```
int kysyArvo (
    char * pPrompti,
    int iArvo )
```

Parameters

<i>pPrompti</i>	Kysymys esim. "Anna haettava numeroarvo: "
<i>iArvo</i>	Arvo, joka halutaan poistaa/etsiä.

Returns

int Palauttaa käyttäjän antaman arvon.

Definition at line 272 of file yleiset.c.

```
272                                     {
273     printf("%s", pPrompti);
274     scanf("%d", &iArvo);
275     getchar();
276     return (iArvo);
277 }
```

4.10.2.3 kysyNimi()

```
char* kysyNimi (
    char * pPrompti,
    char * pNimi )
```

Parameters

<i>pPrompti</i>	Kysymys esim. "Anna kirjoitettavan tiedoston nimi: "
<i>pNimi</i>	Tiedoston nimi/haettava nimi, jota halutaan käyttää.

Returns

char Palauttaa käyttäjän antaman merkkijonon.

Definition at line 258 of file yleiset.c.

```
258                                     {
259     printf("%s", pPrompti);
260     scanf("%s", pNimi);
261     getchar();
262     return (pNimi);
263 }
```

4.10.2.4 listaValikko()

```
int listaValikko (
    void )
```

Parameters

<i>iValinta</i>	Kayttajan antama syote vaihtoehtojen pohjalta.
-----------------	--

Returns

int Palauttaa kayttajan valinnan.

Definition at line 209 of file yleiset.c.

```
209     {
210         int iValinta = 0;
211         printf("\nValitse haluamasi toiminto:\n");
212         printf("1) Lue tiedosto\n");
213         printf("2) Kirjoita tiedosto alusta loppuun\n");
214         printf("3) Kirjoita tiedosto lopusta alkuun\n");
215         printf("4) Tyhjennä taulukko\n");
216         printf("5) Lajittele nousevasti\n");
217         printf("6) Lajittele laskevasti\n");
218         printf("7) Lisää tietoja listaan\n");
219         printf("8) Poista listalta tietoja\n");
220         printf("0) Takaisin\n");
221         printf("Anna valintasi: ");
222         scanf("%d", &iValinta);
223         getchar();
224         return iValinta;
225 }
```

4.10.2.5 mainBinaaripuu()

```
void mainBinaaripuu (
    void )
```

Funktio näyttää valikon ja suorittaa käyttäjän valinnan mukaiset binääripuuhun liittyvät toiminnot.

Definition at line 97 of file yleiset.c.

```
97     {
98         char aNimiLuettava[LEN] = "";
99         char aNimiKirjoitettava[LEN] = "";
100         char aHaettavaNimi[LEN] = "";
101         PUU *pJuuriSolmu = NULL;
102         RBSOLMU *pRBJuuri = NULL;
103         int iValinta = 0;
104         int iArvo = 0;
105
106         do {
107             iValinta = binaaripuuValikko();
108             if (iValinta == 1) {
109                 kysyNimi("Tiedoston nimi: ", aNimiLuettava);
110                 pJuuriSolmu = luoPuu(aNimiLuettava, pJuuriSolmu);
111             } else if (iValinta == 2) {
112                 if (pJuuriSolmu == NULL) {
113                     printf("Luo binääripuu ennen sen kirjoittamista.\n");
114                 } else {
115                     kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
116                     kirjoitaBinaaripuu(aNimiKirjoitettava, pJuuriSolmu);
117                 }
118             }
119             } else if (iValinta == 3) {
120                 if (pJuuriSolmu == NULL) {
121                     printf("Luo binääripuu ennen syvyyshakua.\n");
122                 } else {
123                     }
```

```

124         iArvo = kysyArvo("Anna haettava numeroarvo: ", iArvo);
125         kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
126         tarkistaLoytyykoSyvyyshaula(aNimiKirjoitettava, pJuuriSolmu, iArvo);
127     }
128
129     } else if (iValinta == 4) {
130         if (pJuuriSolmu == NULL) {
131             printf("Luo binääripuu ennen leveyshakua.\n");
132         } else {
133             kysyNimi("Anna haettava nimi: ", aHaettavaNimi);
134             kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
135             leveyshaku(aNimiKirjoitettava, pJuuriSolmu, aHaettavaNimi);
136         }
137     }
138     } else if (iValinta == 5) {
139         pJuuriSolmu = vapautaMuistiPuu(pJuuriSolmu);
140         printf("Muisti vapautettu.\n");
141     }
142     } else if (iValinta == 6) {
143         if (pJuuriSolmu == NULL) {
144             printf("Luo binääripuu ennen poistoa.\n");
145         } else {
146             kysyNimi("Anna poistettava nimi tai arvo: ", aHaettavaNimi);
147             pJuuriSolmu = poistaSolmu(aHaettavaNimi, pJuuriSolmu);
148         }
149     }
150     } else if (iValinta == 7) {
151         if (pJuuriSolmu == NULL) {
152             printf("Luo binääripuu ennen binäärihakua.\n");
153         } else {
154             kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
155             iArvo = kysyArvo("Anna haettava numeroarvo: ", iArvo);
156             iArvo = binaarihaku(aNimiKirjoitettava, pJuuriSolmu, iArvo);
157             if (iArvo == 0) {
158                 printf("Hakemaasi arvoa ei löytynyt binääripuusta.\n");
159             }
160         }
161     }
162     } else if (iValinta == 8) {
163         // Aino säätää ja testaa
164         kysyNimi("Anna luettavan tiedoston nimi: ", aNimiLuettava);
165         kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
166         pRBJuuri = luoRBPuu(pRBJuuri, aNimiLuettava);
167         kirjoitaRB(aNimiKirjoitettava, pRBJuuri);
168         pRBJuuri = vapautaMuistiRB(pRBJuuri);
169     }
170     } else if (iValinta == 0) {
171         printf("Palataan takaisin päävalikkoon.\n");
172     }
173     } else {
174         printf("Tuntematon valinta, yritä uudestaan.\n");
175     }
176 } while (iValinta != 0);
177
178 /* Vapautetaan puun varaama muisti varmuuden vuoksi vielä täällä lopussa erikseen, jos
179  * käyttäjä ei muista sitä tehdä. */
180
181 pJuuriSolmu = vapautaMuistiPuu(pJuuriSolmu);
182 return;
183 }

```

4.10.2.6 mainLista()

```

void mainLista (
    void )

```

Funktio näyttää valikon ja suorittaa käyttäjän valinnan mukaiset listaan liittyvät toiminnot.

Definition at line 17 of file yleiset.c.

```

17     {
18         char aNimiLuettava[LEN] = "";
19         char aNimiKirjoitettava[LEN] = "";
20         int iArvo = 0;
21         int iIndeksi = 0;
22         int iValinta = 0;
23         int iSamaLKM = 0;
24         TIEDOT *pAlku = NULL;

```

```

25
26     do {
27         iValinta = listaValikko();
28         if (iValinta == 1) {
29             kysyNimi("Anna luettavan tiedoston nimi: ", aNimiLuettava);
30             pAlku = lue(aNimiLuettava, pAlku);
31
32         } else if (iValinta == 2) {
33             if (pAlku == NULL) {
34                 printf("Lue tiedosto ennen kirjoittamista.\n");
35             } else {
36                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
37                 kirjoitaTiedostoAlustaLoppuun(aNimiKirjoitettava, pAlku);
38             }
39
40         } else if (iValinta == 3) {
41             if (pAlku == NULL) {
42                 printf("Lue tiedosto ennen kirjoittamista.\n");
43             } else {
44                 kysyNimi("Anna kirjoitettavan tiedoston nimi: ", aNimiKirjoitettava);
45                 kirjoitaTiedostoLopustaAlkuun(aNimiKirjoitettava, pAlku);
46             }
47
48         } else if (iValinta == 4) {
49             pAlku = vapautaMuisti(pAlku);
50             printf("Muisti vapautettu.\n");
51
52         } else if (iValinta == 5) {
53             pAlku = lomitustaLajittelu(pAlku);
54             printf("Numerot lajiteltu nousevasti.\n");
55
56         } else if (iValinta == 6) {
57             // Laskeva lajittelu
58
59         } else if (iValinta == 7) {
60             iIndeksi = kysyArvo("Mihin kohtaan listaa haluat lisätä tietoja (indeksi): ", iIndeksi);
61             kysyNimi("Anna lisättävä nimi: ", aNimiKirjoitettava);
62             iArvo = kysyArvo("Anna tämän nimisten lukumäärä: ", iArvo);
63             pAlku = lisaaListaan(pAlku, iIndeksi, aNimiKirjoitettava, iArvo);
64
65         } else if (iValinta == 8) {
66             // Poistaminen listalta
67             iArvo = kysyArvo("Anna poistettava lukumäärä: ", iArvo);
68             iSamaLKM = useammallaAlkiollaSamaLKM(pAlku, iArvo);
69             if (iSamaLKM == 1) {
70                 pAlku = poistaListastaLkmPerusteella(pAlku, iArvo);
71             } else if (iSamaLKM > 1) {
72                 kysyNimi("Anna poistettava nimi: ", aNimiKirjoitettava);
73                 pAlku = poistaListastaNimenPerusteella(pAlku, aNimiKirjoitettava);
74             }
75
76         } else if (iValinta == 0) {
77             printf("Palataan takaisin päävalikkoon.\n");
78
79         } else {
80             printf("Tuntematon valinta, yritä uudestaan.\n");
81         }
82     } while (iValinta != 0);
83
84     /* Vapautetaan listan varaama muisti varmuuden vuoksi vielä täällä lopussa erikseen, jos
85     * käyttäjä ei muista sitä tehdä. */
86
87     pAlku = vapautaMuisti(pAlku);
88     return;
89 }

```

4.10.2.7 valikko()

```

int valikko (
    void )

```

Parameters

<i>iValinta</i>	Kayttajan antama syöte.
-----------------	-------------------------

Returns

int Palauttaa kayttajan valinnan.

Definition at line 191 of file yleiset.c.

```
191     {
192         int iValinta = 0;
193         printf("Valitse haluamasi toiminto:\n");
194         printf("1) Lista\n");
195         printf("2) Binääripuu\n");
196         printf("0) Lopeta\n");
197         printf("Anna valintasi: ");
198         scanf("%d", &iValinta);
199         getchar();
200         return iValinta;
201     }
```

